

- nivel de bits, `|`, puede ser utilizada para combinar varias opciones en un valor único `long`. Llamar la función miembro `flags` para un flujo y especificar estas opciones con la función `OR` define las opciones de dicho flujo y devuelve un valor `long` que contiene las opciones precedentes. A menudo, este valor es guardado, por lo que `flags` puede ser llamada con este valor guardado, a fin de restaurar las opciones de flujo anteriores.
- La función `flags` debe definir un valor que represente los ajustes totales de todas las banderas. La función `setf` con un argumento, por otra parte, hace la operación a nivel de bit “OR” correspondiente a las banderas especificadas con los ajustes existentes de bandera, para formar un nuevo estado de formato.
 - La bandera `showpoint` se define para obligar a un número de punto flotante a que sea extraído con un punto decimal y una cantidad de dígitos significativos según esté definido en la precisión.
 - Las banderas `left` y `right` hacen que los campos queden justificados a la izquierda con caracteres de relleno a la derecha, o a la derecha con caracteres de relleno a la izquierda, respectivamente.
 - La bandera `internal` indica que el signo de un número (o la base cuando se trate de la bandera `ios::showbase`) deberá quedar justificado a la izquierda dentro de un campo, la magnitud justificada a la derecha, y llenar los espacios intermedios con un carácter de relleno.
 - `ios::adjustfield` contiene las banderas `left`, `right`, e `internal`.
 - La función miembro `fill` define el carácter de relleno a utilizar con los campos ajustados `left`, `right`, e `internal` (un espacio en blanco es el valor por omisión); el carácter de relleno anterior es devuelto. El manipulador de flujo `setfill` también define el carácter de relleno.
 - El miembro estático `ios::basefield` incluye los bits `oct`, `hex`, y `dec` a fin de especificar que los enteros deben ser tratados como valores octal, hexadecimal y decimales, respectivamente. Si no está activo ninguno de estos bits, la salida de enteros por omisión se hace en decimal; las extracciones de flujo procesan los datos en la misma forma en la cual fueron suministrados.
 - Defina la bandera `showbase` para obligar la base de la extracción de un valor integral.
 - El miembro de datos estático `ios::floatfield` contiene las banderas `scientific` y `fixed`. Defina la bandera `scientific` para extraer un número en punto flotante en formato científico. Defina la bandera `fixed` para extraer un número de punto flotante con la precisión especificada por la función miembro `precision`.
 - La llamada `cout.setf(0, ios::floatfield)` restaura el formato por omisión para la exhibición de los números de punto flotante.
 - Active la bandera `uppercase` para obligar a la extracción de una `X` o `E` mayúsculas con los enteros hexadecimales o en la notación científica de valores de punto flotante, respectivamente. Cuando esté activa, la bandera `ios::uppercase` hará que todas las letras en un valor hexadecimal aparezcan en mayúsculas.
 - La función miembro `flags` sin argumento, devuelve el valor `long` de los ajustes actuales de las banderas de formato. La función miembro `flags` con el argumento `long` define las banderas de formato especificadas por el argumento y devuelve los ajustes anteriores de bandera.
 - La función miembro `setf` define las banderas de formato en su argumento y devuelve los ajustes de banderas anteriores como un valor `long`.

- La función miembro `setf` con dos argumentos `long` —un bit `long` y un campo de bits `long`— elimina los bits en el campo de bits, y a continuación ajusta el bit del primer argumento.
- La función miembro `unsetf` restaura las banderas designadas y devuelve el valor de las banderas antes de su restauración.
- El manipulador de flujo parametrizado `setiosflags` ejecuta las mismas funciones que la función miembro `flags`.
- El manipulador de flujo parametrizado `resetiosflags` ejecuta las mismas funciones que la función miembro `unsetf`.
- El estado de un flujo puede ser probado mediante bits en la clase `ios`.
- El `eofbit` es activado para un flujo de entrada cuando se encuentra un fin de archivo durante una operación de entrada. La función miembro `eof` se utiliza para determinar si `eofbit` está activo.
- El `failbit` se activa para un flujo cuando en dicho flujo ocurrió un error de formato, pero sin pérdida de caracteres. La función miembro `fail` determina si una operación de flujo ha fallado; por lo general, es posible recuperarse de dichos errores.
- `badbit` se activa en un flujo cuando ha ocurrido un error cuyo resultado es pérdida de datos. La función miembro `bad` determina si una operación de flujo ha fallado. Estas fallas serias, por lo regular no son recuperables.
- La función miembro `good` devuelve verdadero si las funciones `bad`, `fail` y `eof` todas ellas devuelven falso. Las operaciones de entrada/salida únicamente deberían ser llevadas a cabo sobre flujos “buenos”.
- La función miembro `rdstate` devuelve el estado de error del flujo.
- La función miembro `clear` se utiliza por lo regular para regresar el estado del flujo a “bueno”, de tal forma que se pueda proceder con entradas y salidas sobre dicho flujo.
- El usuario puede hacer la homonimia de los operadores de inserción y de extracción de flujo para llevar a cabo entradas/salidas para tipos definidos por usuario.
- El operador de extracción de flujo homónimo toma como argumento una referencia a `istream` y una referencia a un tipo definido por usuario, y devuelve una referencia a `istream`.
- El operador de inserción de flujo homónimo toma como referencias una referencia a `ostream` y una referencia a un tipo definido por usuario, y devuelve una referencia a `ostream`.
- A menudo, se declaran funciones homónimas `operator` como funciones `friend` a una clase; esto permite el acceso a miembros de clase no públicos.
- C++ proporciona la función miembro `tie` para sincronizar operaciones `istream` y `ostream` para asegurarse que las salidas aparezcan antes de entradas subsecuentes.

Terminología

función miembro `bad`
`badbit`
`cerr`
`cin`
 función miembro `clear`

`clog`
`cout`
 manipulador de carácter de flujo `dec`
 carácter de llenado por omisión (espacio en blanco)
 precisión por omisión

fin de archivo	<code>left</code>
<code>endl</code>	justificado a la izquierda
función miembro <code>eof</code>	manipulador de flujo <code>oct</code>
<code>eofbit</code>	clase <code>ofstream</code>
extensibilidad	función miembro <code>operator!</code>
función miembro <code>fail</code>	función miembro <code>operator void *</code>
<code>failbit</code>	clase <code>ostream</code>
ancho de campo	relleno
carácter de llenado	manipulador de flujo parametrizado
función miembro <code>fill</code>	función miembro <code>peek</code>
función miembro <code>flags</code>	función miembro <code>precision</code>
función miembro <code>flush</code>	flujo predeterminado
manipulador de flujo <code>flush</code>	función miembro <code>put</code>
banderas de formato	función miembro <code>putback</code>
estados de formato	función miembro <code>rdbuf</code>
entrada/salida con formato	función miembro <code>read</code>
clase <code>fstream</code>	manipulador de flujo <code>resetiosflags</code>
función miembro <code>gcount</code>	justificado a la derecha
función miembro <code>get</code>	manipulador de flujo <code>setbase</code>
función miembro <code>getline</code>	función miembro <code>setf</code>
función miembro <code>good</code>	manipulador de flujo <code>setfill</code>
manipulador de flujo <code>hex</code>	manipulador de flujo <code>setiosflags</code>
clase <code>ifstream</code>	manipulador de flujo <code>setprecision</code>
función miembro <code>ignore</code>	manipulador de flujo <code>setw</code>
formato en núcleo	<code>skipws</code>
formato en memoria	bibliotecas de clases de flujo
archivo de cabecera estándar <code><iomanip.h></code>	operador de extracción de flujo (<code>>></code>)
clase <code>ios</code>	entrada de flujo
<code>ios::adjustfield</code>	operador de inserción de flujo (<code><<</code>)
<code>ios::basefield</code>	manipulador <code>stream</code>
<code>ios::fixed</code>	salida de flujo
<code>ios::floatfield</code>	función miembro <code>tie</code>
<code>ios::internal</code>	entrada/salida de tipo seguro
<code>ios::scientific</code>	entrada/salida sin formato
<code>ios::showbase</code>	función miembro <code>unsetf</code>
<code>ios::showpoint</code>	<code>uppercase</code>
<code>ios::showpos</code>	flujos definidos por usuario
clase <code>iostream</code>	caracteres de espacio en blanco
clase <code>istream</code>	<code>width</code>
0 a la derecha (octal)	función miembro <code>write</code>
0x ó 0X a la izquierda (hexadecimal)	función miembro <code>ws</code>

Errores comunes de programación

- 21.1 Intentar leer de un `ostream` (o de cualquier otro flujo de sólo salida).
- 21.2 Intentar escribir a un `istream` (o a cualquier otro flujo de sólo entrada).
- 21.3 Omitir paréntesis para obligar a una correcta precedencia al utilizar los operadores de inserción de flujo `<<` o de extracción de flujo `>>` que tienen una relativamente alta precedencia.
- 21.4 Cuando no se de un campo lo suficiente amplio para manejar salidas, dichas salidas se imprimirán del ancho que requieran, posiblemente causando salidas erróneas o difíciles de leer.

Prácticas sanas de programación

- 21.1 En programas C++, utilice exclusivamente la forma de entradas/salidas de C++, a pesar del hecho que para los programadores C++ esté disponible el estilo de entradas/salidas de C.
- 21.2 Al extraer expresiones, colóquelas entre paréntesis, para evitar problemas de precedencia de operadores entre los operadores de la expresión y el operador `<<`.

Sugerencia de rendimiento

- 21.1 Utilice entradas/salidas sin formato para un rendimiento máximo en procesos de alto volumen de archivos.

Observaciones de ingeniería de software

- 21.1 El estilo de entradas/salidas de C++ es de tipo seguro.
- 21.2 C++ permite un tratamiento común de entradas/salidas de tipos predefinidos y de tipos definidos por usuario. Este tipo de estado común facilita el desarrollo de software en general y de la reutilización de software en particular.
- 21.3 Se pueden añadir a C++ nuevas capacidades de entrada/salida para tipos definidos por usuario, sin modificar las declaraciones o los miembros de datos privados, ya sea para la clase `ostream` o para la clase `istream`. Esto fomenta la extensibilidad del lenguaje de programación C++ —lo que es uno de los aspectos más atractivos de C++.

Ejercicios de autoevaluación

- 21.1 Llene cada uno de los siguientes espacios vacíos:
- Las funciones de operador de flujo homónimas deben ser definidas como funciones _____ de una clase.
 - Los bits de formato de justificación que se pueden definir son _____, _____, y _____.
 - En C++ la entrada/salida ocurre como un _____ de bytes.
 - Los manipuladores de flujo parametrizados _____, y _____ se pueden utilizar para activar y desactivar banderas de estado de formato.
 - La mayor parte de los programas de C++ deben incluir el archivo de cabecera _____ que contiene información básica requerida para todas las operaciones de flujo de entrada/salida.
 - Las funciones miembro _____, y _____ pueden ser utilizadas para activar y reactivar banderas de estado de formato.
 - El archivo de cabecera _____ contiene información para llevar a cabo el formato “en memoria”.
 - Al utilizar los manipuladores parametrizados, se debe incluir en el programa el archivo de cabecera _____.
 - El archivo de cabecera _____ contiene información para llevar a cabo procesamiento de archivos controlados por usuario.
 - El manipulador de flujo _____ inserta un carácter de nueva línea en el flujo de salida y vacía el flujo de salida.
 - El archivo de cabecera _____ contiene información para aquellos programas que mezclan entradas y salidas en estilo C y en estilo C++.
 - La función miembro `ostream` _____ se utiliza para llevar a cabo salidas sin formato.
 - Las operaciones de entrada son soportadas por la clase _____.
 - Las salidas al flujo de error estándar son dirigidas ya sea por el objeto de flujo _____ o _____.
 - Las operaciones de salida están soportadas por la clase _____.
 - El símbolo para el operador de inserción de flujo es _____.

- q) Los cuatro objetos que corresponden a los dispositivos estándar en el sistema incluyen _____, _____, y _____.
- r) El símbolo para operador de extracción de flujo es _____.
- s) Los manipuladores de flujo _____, _____, y _____ se utilizan para especificar que los enteros deben ser mostrados en formatos octal, hexadecimal y decimal, respectivamente.
- t) La precisión por omisión para mostrar valores en punto flotante es _____.
- u) Cuando está activa, la bandera _____ hace que los números positivos numéricos aparezcan con un signo más.

21.2 Indique si lo que sigue es verdadero o falso. Si la respuesta es falsa, explique por qué.

- a) La función miembro de flujo `flags()` con un argumento `long`, define a su argumento a la variable de estado `flags` y devuelve su valor anterior.
- b) El operador de inserción de flujo `<<` y el operador de extracción de flujo `>>` son homónimos para manejar todos los tipos de datos estándar —incluyendo cadenas y direcciones de memoria, así como todos los tipos de datos definidos por usuario.
- c) La función miembro de flujo `flags()` sin argumentos, restaura todos los bits de bandera en la variable de estado `flags`.
- d) Se puede hacer la homonimia del operador de extracción de flujo `>>` con una función operador que toma como argumentos una referencia `istream` y una referencia a un tipo definido por usuario, y devuelve una referencia `istream`.
- e) El manipulador de flujo `ws` pasa por alto el espacio en blanco a la izquierda de un flujo de entrada.
- f) Se puede hacer la homonimia del operador de inserción de flujo `<<` con una función operador que toma como referencia una referencia a `istream` y una referencia a un tipo definido por usuario y devuelve una referencia `istream`.
- g) La entrada con el operador de extracción de flujo `>>` siempre pasará por alto los caracteres de espacio en blanco del flujo de entrada.
- h) Las características de entrada y de salida son parte de C++.
- i) La función miembro de flujo `rdstate()` devuelve el estado del flujo actual.
- j) El flujo `cout` por lo general, está conectado a la pantalla de exhibición.
- k) La función miembro de flujo `good()` devuelve verdadero si las funciones miembro `bad()`, `fail()` y `eof()` todas ellas devuelven falso.
- l) El flujo `cin` por lo regular está conectado a la pantalla de exhibición.
- m) Si ocurre un error no recuperable durante una operación de flujo, la función miembro `bad()` devolverá verdadero.
- n) La salida a `cerr` no tiene búfer y la salida a `clog` sí lo tiene.
- o) Cuando está activa la bandera `ios::showpoint`, los valores en punto flotante están forzados a imprimirse con los seis dígitos de precisión por omisión —siempre y cuando el valor de la precisión no haya sido modificado, en cuyo caso los valores de punto flotante se imprimirán con la precisión especificada.
- p) La función miembro `ostream` de nombre `put` extrae el número especificado de caracteres.
- q) Los manipuladores de flujo `dec`, `oct` y `hex` solamente afectan la siguiente operación de salida de enteros.
- r) Al ser extraído, por omisión las direcciones de memoria se exhiben como enteros `long`.

21.3 Escriba un solo enunciado que ejecute la tarea indicada para cada uno de los siguientes.

- a) Extraiga la cadena `"Enter your name: "`.
- b) Active una bandera para que el exponente en notación científica y las letras en los valores hexadecimales se impriman en mayúsculas.
- c) Extraiga la dirección de la variable `string` del tipo `char *`.
- d) Active una bandera, de tal forma que los valores de punto flotante se impriman en notación científica.

- e) Extraiga la dirección de la variable `integerPtr` del tipo `int *`.
- f) Active una bandera, de tal forma que cuando valores enteros sean extraídos se exhiban la base entera para valores octal y hexadecimal.
- g) Extraiga el valor al cual apunta `floatPtr` del tipo `float *`.
- h) Utilice una función miembro de flujo para definir el carácter de llenado como `' '` para imprimir en anchos de campo mayores que los valores siendo extraídos. Escriba un enunciado por separado para ello mediante un manipulador de flujo.
- i) Extraiga los caracteres `'O'` y `'K'` en un enunciado mediante la función `put` de `ostream`.
- j) Obtenga el siguiente carácter en el flujo de entrada sin extraerlo del mismo flujo.
- k) Introduzca un solo carácter en la variable `c` de tipo `char` utilizando la función miembro `istream::get` de dos formas distintas.
- l) Introduzca y descarte los siguientes seis caracteres del flujo de entrada.
- m) Utilice la función miembro `read` de `istream` para introducir 50 caracteres al arreglo `line` del tipo `char`.
- n) Lea 10 caracteres en el arreglo de caracteres `name`. Si se encuentra el delimitador `'.'` detenga la lectura de caracteres. No retire el delimitador del flujo de entrada. Escriba otro enunciado que lleve a cabo esa tarea y elimine el delimitador del flujo de entrada.
- o) Utilice la función miembro `gcount` de `istream` para determinar el número de caracteres introducidos en el arreglo de caracteres `line` por la última llamada a la función miembro `read` de `istream` y extraiga dicho número de caracteres utilizando la función miembro `write` de `ostream`.
- p) Escriba enunciados por separado para vaciar el flujo de salida utilizando una función miembro y un manipulador de flujo.
- q) Extraiga los siguientes valores: `124`, `18.376`, `'Z'`, `1000000`, y `"string"`.
- r) Imprima el ajuste de precisión actual utilizando una función miembro.
- s) Introduzca un valor entero en la variable `months` de `int` y un valor de punto flotante en la variable `percentageRate` de `float`.
- t) Imprima `1.92`, `1.925`, y `1.9258` con 3 dígitos de precisión utilizando un manipulador.
- u) Imprima el entero `100` en base octal, hexadecimal y decimal utilizando manipuladores de flujo.
- v) Imprima el entero `100` en base decimal, octal y hexadecimal utilizando un solo manipulador de flujo para modificar la base.
- w) Imprima `1234` justificado a la derecha en un campo de 10 dígitos.
- x) Lea los caracteres al arreglo de caracteres `line` hasta que se encuentre con el carácter `'z'` hasta un límite de 20 caracteres (incluyendo el carácter de terminación null). No extraiga el carácter del delimitador del flujo.
- y) Utilice las variables enteras `x` e `y` para especificar el ancho de campo y la precisión utilizada para mostrar el valor `87.4573` de tipo `double` y exhiban dicho valor.

21.4 Identifique el error en cada uno de los enunciados siguientes y explique cómo corregirlo.

- a) `cout << "Value of x <= y is: " << x <= y;`
- b) El siguiente enunciado debe de imprimir el valor entero de `'c'`
`cout << 'c';`
- c) `cout << "A string in quotes";`

21.5 Muestre la salida para cada uno de los siguientes:

- a) `cout << "12345\n";`
`cout.width(5);`
`cout.fill('*');`
`cout << 123 << '\n' << 123;`
- b) `cout << setw(10) << setfill('$') << 10000;`
- c) `cout << setw(8) << setprecision(3) << 1024.987654;`

```
d) cout << setiosflags (ios::showbase) << oct << 99
   << '\n' << hex 99;
e) cout << 100000 << '\n'
   << setiosflags (ios::showpos) << 100000;
f) cout << setw(10) << setprecision(2) <<
   << setiosflags(ios::scientific) << 444.93738;
```

Respuestas a los ejercicios de autoevaluación

21.1 a) friend. b) `ios::left`, `ios::right`, y `ios::internal`. c) flujos. d) `setiosflags`, `resetiosflags`. e) `iostream.h`. f) `setf`, `unsetf`. g) `strstream.h`. h) `iomanip.h`. i) `fstream.h`. j) `endl`. k) `stdiostream.h`. l) `write`. m) `istream`. n) `cerr` o `clog`. o) `ostream`. p) `<<`. q) `cin`, `cout`, `cerr`, y `clog`. r) `>>`. s) `oct`, `hex`, `dec`. t) seis dígitos de precisión. u) `ios::showpos`.

- 21.2 a) Verdadero.
 b) Falso. No se hace la homonimia de los operadores de extracción y de inserción de flujo para todos los tipos definidos por usuario. El programa de una clase debe proporcionar en forma específica las funciones de operador homónimas para hacer la homonimia de los operadores de flujo para uso con cada uno de los tipos definidos por usuario.
 c) Falso. La función miembro de flujo `flags()` sin argumento, sólo devuelve el valor actual de la variable de estado `flags`.
 d) Verdadero.
 e) Verdadero.
 f) Falso. Para hacer la homonimia del operador de inserción de flujo `<<`, la función de operador homónima debe tomar como argumentos una referencia `ostream` y una referencia a un tipo definido por usuario, y devolver una referencia `ostream`.
 g) Verdadero. A menos de que `ios::skipws` esté desactivado.
 h) Falso. Las características de entrada y de salida de C++ se proporcionan como parte de la biblioteca estándar de C++. El lenguaje de C++ no contiene capacidades para el procesamiento de entradas, salidas o de archivos.
 i) Verdadero.
 j) Verdadero.
 k) Verdadero.
 l) Falso. El flujo `cin` está conectado a la entrada estándar de la computadora, que por lo regular es el teclado.
 m) Verdadero.
 n) Verdadero.
 o) Verdadero.
 p) Falso. La función miembro `put` de `ostream` extrae su único argumento de carácter.
 q) Falso. Los manipuladores de flujo `dec`, `oct` y `hex` establecen el estado de formato de salida para enteros a la base especificada, hasta que ésta sea otra vez específicamente modificada o el programa se termine.
 r) Falso. Por omisión las direcciones de memoria se exhiben en formato hexadecimal. Para mostrar las direcciones como enteros `long`, la dirección deberá convertirse explícitamente (cast) a un valor `long`.

21.3 a) `cout << "Enter your name: ";`
 b) `cout.setf(ios::uppercase);`
 c) `cout << long(string);`
 d) `cout.setf(ios::scientific, ios::floatfield);`
 e) `cout << integerPtr;`
 f) `cout << setiosflags(ios::showbase);`

```
g) cout << *floatPtr;
h) cout.fill('*');
   cout << setfill('*');
i) cout.put('0').put('K');
j) cin.peek();
k) c = cin.get();
   cin.get(c);
l) cin.ignore(6);
m) cin.read(line,50);
n) cin.get(name, 10, '.');
   cin.getline(name, 10, '.');
o) cout.write(line, cin.gcount());
p) cout.flush();
   cout << flush;
q) cout << 124 << 18.376 << 'Z' << 1000000 << "String";
r) cout.precision();
s) cin >> months >> percentageRate;
t) cout << setprecision(3) << 1.92 << '\t'
   << 1.925 << '\t' << 1.9258;
u) cout << oct << 100 << hex << 100 << dec << 100;
v) cout << 100 << setbase(8) << 100 << setbase(16) << 100;
w) cout << setw(10) << 1234;
x) cin.get(line, 20, 'z');
y) cout << setw(x) << setprecision(y) << 87.4573;
```

- 21.4 a) Error: es mayor la precedencia del operador `<<` que la del operador `<=`, lo que hace que se evalúe en forma incorrecta el enunciado y causará que se emita un error por el compilador. Corrección: para corregir el enunciado, añada paréntesis encerrando la expresión `x <= y`. Este problema ocurrirá con cualquier expresión que utilice operadores de una precedencia menor que el operador `<<`, si la expresión no se encierra entre paréntesis.
 b) Error: en C++, los caracteres no se tratan como pequeños enteros como es el caso en C. Corrección: imprimir el valor numérico de un carácter en el conjunto de caracteres de la computadora, el carácter deberá ser convertido explícitamente (cast) a un valor entero como en el siguiente:

```
cout << int('c');
```

- c) Error: los caracteres entre comillas no pueden ser impresos como una cadena, a menos de que se utilice una secuencia de escape. Corrección: imprima la cadena en alguna de las siguientes formas:
`cout << "" << "A string in quotes" << "";`
`cout << "\"A string in quotes\"";`

21.5 a) 12345
 **123
 123
 b) \$\$\$\$10000
 c) 1024.988
 d) 0143
 0x63
 e) 100000
 +100000
 f) 4.45e+02

Ejercicios

21.6 Escriba un enunciado para cada uno de los siguientes:

- Imprima el entero 40000 justificado a la izquierda en un campo de 15 dígitos.
- Lea una cadena a la variable de arreglo de caracteres `state`.
- Imprima 200 con y sin signo.
- Imprima 100 en forma hexadecimal precedido por 0x.
- Lea caracteres al arreglo `s` hasta que se encuentre con el carácter 'p' hasta un límite de 10 caracteres (incluyendo el carácter nulo de terminación). Extraiga el delimitador del flujo de entrada y descártelo.
- Imprima 1.234 en un campo de 9 dígitos con ceros a la izquierda.
- Lea un carácter de la forma "characters" de la entrada estándar. Almacene la cadena en el arreglo de caracteres `s`. Elimine las comillas del flujo de entrada. Lea un máximo de 50 caracteres (incluyendo el carácter nulo de terminación).

21.7 Escriba un programa para probar los valores enteros de entrada en formato decimal, octal y hexadecimal. Extraiga cada entero leído por el programa en los tres formatos. Pruebe el programa con los siguientes datos de entrada: 10, 010, 0x10.

21.8 Escriba un programa que imprima los valores de apuntador utilizando conversiones explícitas (cast) para todos los tipos de datos enteros. ¿Cuáles son los que imprimen valores raros? ¿Cuáles causan errores?

21.9 Escriba un programa para probar los resultados de imprimir el valor entero 12345 y el valor de punto flotante 1.2345 en campos de varios tamaños. ¿Qué ocurre cuando los valores se imprimen en campos que contienen menos dígitos que los valores?

21.10 Escriba un programa que imprima el valor 100.453627 redondeado al siguiente dígito, décimo, centésimo, milésimo y diezmilésimo.

21.11 Escriba un programa que introduzca una cadena desde el teclado y que determine la longitud de dicha cadena. Imprima la cadena utilizando como ancho de campo el doble de su longitud.

21.12 Escriba un programa que convierta temperaturas enteras Fahrenheit desde 0 hasta 212 grados a temperaturas Celsius en punto flotante con 3 dígitos de precisión. Utilice la fórmula.

```
celsius = 5.0/9.0 * (fahrenheit - 32);
```

para efectuar el cálculo. La salida deberá ser impresa en dos columnas justificadas a la derecha, y las temperaturas Celsius deberán estar precedidas por un signo, tanto para valores positivos como negativos.

21.13 En algunos lenguajes de programación, las cadenas se introducen encerradas ya sea entre comillas sencillas o dobles. Escriba un programa que lea tres cadenas `suzy`, "suzy", y 'suzy'. Las comillas sencillas y dobles ¿son ignoradas o son leídas como parte de la cadena?

21.14 En la figura 21.30 los operadores de extracción y de inserción de flujo son homónimos para la entrada y salida de objetos `PhoneNumber`. Vuelva a escribir el operador de extracción de flujo para llevar a cabo las siguientes tareas de verificación de error a la entrada. Note que la función `operator>>` necesitará recodificarse totalmente.

- Introduzca en un arreglo la totalidad del número telefónico. Pruebe que han sido introducidos todos los números telefónicos (debe haber un total de 14 caracteres para un número telefónico de la forma (800) 555-1212). Use la función miembro de flujo `clear` para definir el bit `ios::fail` para una entrada incorrecta.
- En un número telefónico el código de área y la centralilla no empiezan con 0 o con 1. Pruebe el primer dígito de las porciones del código de área y de centralilla del número telefónico, para estar seguro que ninguno empieza con 0 o con 1. Utilice la función miembro de flujo `clear` para definir el bit `ios::fail` en caso de entrada incorrecta.

- El dígito intermedio de un código de área es siempre 0 ó 1. Pruebe el dígito intermedio de esta porción buscando un valor de 0 ó 1. Utilice la función miembro de flujo `clear` para definir el bit `ios::fail` en caso de entrada incorrecta. Si ninguna de las anteriores operaciones resulta en la activación del bit `ios::fail` debido a entrada incorrecta, copie las tres porciones del número telefónico en los miembros `areaCode`, `exchange`, y `line` del objeto `PhoneNumber`. En el programa principal, si el bit `ios::fail` ha sido activado en la entrada, haga que el programa imprima un mensaje de error y termine antes de imprimir el número telefónico.

21.15 Escriba un programa que lleve a cabo cada uno de los siguientes:

- Crear una clase definida por usuario de nombre `point` que contenga los miembros de dato enteros privados `xCoordinate` y `yCoordinate`, y declarar las funciones de operador homónimas de extracción y de inserción de flujo como `friends` de la clase.
- Defina las funciones de operador de inserción y de extracción de flujo. La función de operador de extracción de flujo debe determinar si los datos introducidos son válidos, de lo contrario, deberá activar `ios::failbit` para indicar entrada incorrecta. Una vez que haya ocurrido el error de entrada el operador de inserción de flujo no deberá ser capaz de mostrar el punto.
- Escriba una función `main` que pruebe la entrada y la salida de la clase definida por usuario `point` utilizando los operadores de inserción y de extracción de flujo homónimos.

21.16 Escriba un programa que lleve a cabo cada uno de los siguientes:

- Crear la clase definida por usuario `complex`, que contenga los miembros de dato enteros privados `real` e `imaginary`, y que declare las funciones de operador de extracción y de inserción de flujo homónimas como `friends` de la clase.
- Defina las funciones de operador o de extracción y de inserción de flujo. La función de operador de extracción de flujo deberá determinar si los datos introducidos son datos válidos, y de lo contrario, deberá activar `ios::failbit` para indicar entrada incorrecta. La entrada deberá ser de la forma

```
3 + 8i
```

Los valores pueden ser negativos o positivos, y es posible que alguno de los dos valores no sea proporcionado. Si uno de ellos no es proporcionado, el miembro de datos apropiado deberá ser definido como 0. Si ha ocurrido un error de entrada el operador de inserción de flujo no debe ser capaz de mostrar el punto. El formato de salida debe ser idéntico al formato de entrada que se muestra arriba. Para los valores imaginarios negativos, deberá imprimirse un signo menos en vez de un signo más.

- Escriba una función `main` que pruebe la entrada y la salida de la clase definida por usuario `complex` utilizando los operadores de inserción y de extracción de flujo homónimos.

21.17 Escriba un programa que utilice una estructura `for` para imprimir una tabla de valores ASCII para los caracteres en el conjunto de caracteres ASCII desde 33 hasta 126. El programa deberá imprimir el valor decimal, el valor octal, el hexadecimal y el valor de carácter de cada uno de los caracteres. Utilice los manipuladores de flujo `dec`, `oct` y `hex` para imprimir los valores enteros.

21.18 Escriba un programa para demostrar que las funciones miembro `istream` de nombre `getline` y `get` de tres argumentos cada una de ellas termina la cadena de entrada con un carácter nulo de terminación de cadena. También demuestre que `get` deja el carácter delimitador en el flujo de entrada, en tanto que `getline` extrae el carácter delimitador y lo descarta. ¿Qué ocurre con los caracteres no leídos dentro del flujo?

21.19 Escriba un programa que cree el manipulador definido por usuario `skipwhite` para pasar por alto los caracteres de espacio en blanco a la izquierda en el flujo de entrada. El manipulador deberá utilizar la función `isspace` de la biblioteca `cctype.h` para probar si el carácter es un carácter de espacio en blanco. Cada carácter deberá ser introducido utilizando la función miembro `get` de `istream`. Cuando se encuentre

con un carácter distinto a un espacio en blanco, el manipulador `skipwhite` termina su trabajo colocando el carácter de regreso en el flujo de entrada y devolviendo una referencia `istream`.

Pruebe el manipulador definido por usuario creando una función `main` en el cual quede la bandera `ios::skipws` desactivada de tal forma que el operador de extracción de flujo no se salte en forma automática los espacios en blanco. A continuación pruebe el manipulador sobre el flujo de entrada introduciendo un carácter precedido por un espacio en blanco como entrada. Imprima el carácter que fue introducido a fin de confirmar que el carácter de espacio en blanco no fue introducido.

Bibliografía

- (A193) Allison, C., "Code Capsules: A C++ Date Class, Parte I" *The C Users Journal*, Vol 11, No. 2, Febrero 1993, pp. 123-131.
- (An92) Anderson, A. E., y W. J. Heinze, *C++ Programming and Fundamental Concepts*, Englewood Cliffs, NJ: Prentice Hall, 1992.
- (Ba93) Bar-David, T., *Object-Oriented Design for C++*, Englewood Cliffs, NJ: Prentice Hall, 1993.
- (Be93) Berard, E. V., *Essays on Object-Oriented Software Engineering: Volumen I*, Englewood Cliffs, NJ: Prentice Hall, 1993.
- (Br91) Borland, *Borland C++ Programmer's Guide*, Parte No. 14MN-TCP04, Scotts Valley, CA: Borland International, Inc., 1991.
- (Br91a) Borland, *Borland C++ Getting Started*, Parte No. 14MN-TCP02, Scotts Valley, CA: Borland International, Inc., 1991.
- (Br91b) Borland, *Borland C++ 3.0 Programmers Guide*, Scotts Valley, CA: Borland International, Inc., 1991.
- (By93) Byron, D., "The Case for Object Technology Standards," *CASE Trends*, Septiembre de 1993, pp. 22-26.
- (Co93) Computerworld, "The CW Guide to Object-Oriented Programming" *Computerworld*, Junio 14, 1993, pp. 107-130.
- (De90) Deitel, H. M., *Operating Systems*, Second Edition, Reading, MA: Addison-Wesley, 1990.
- (El90) Ellis, M. A. y B. Stroustrup, *The Annotated C++ Reference Manual*, Reading, MA: Addison-Wesley, 1990.
- (Fl93) Flaming, B., *Practical Data Structures in C++*, John Wiley & Sons, 1993.
- (Ha93) Hagan, T., "C++ Class Libraries for GUIs," *Open Systems Today*, Febrero 15, 1993, pp. 54, 58.
- (Ja93) Jacobson, I., "Is Object Technology Software's Industrial Platform?" *IEEE Software Magazine*, Vol. 10, No. 1, Enero de 1993, pp. 24-30.
- (Ko93) Kozaczynski, W., y A. Kuntzmann-Combelle, "What It Takes to Make OO Work," *IEEE Software Magazine*, Vol.10, No.1, Enero 1993, pp. 20-23.
- (Li91) Lippman, S. B., *C++ Primer* (Second Edition). Reading, MA: Addison-Wesley Publishing Company, 1991.
- (Lo93) Lorenz, M., *Object-Oriented Software Development: A Practical Guide*, Englewood Cliffs, NJ: Prentice Hall, 1993.
- (Lu92) Lucas, P. J., *The C++ Programmer's Handbook*, Englewood Cliffs, NJ: Prentice Hall, 1992.
- (Ma93) Martin, J., *Principles of Object-Oriented Analysis and Design*, Englewood Cliffs, NJ: Prentice Hall, 1993.
- (Me93) Matsche, J.J., "Object-oriented programming in Standard C," *Object Magazine*, Vol.2, No.5, Enero/Febrero 1993, pp. 71-74.

- (Mi91) Microsoft, *Microsoft C/C++ Class Libraries Reference* (Versión 7.0), Redmond, WA: Microsoft Corporation, 1991.
- (Mi91a) Microsoft, *Microsoft C/C++ C++ Language Reference* (Versión 7.0), Redmond, WA: Microsoft Corporation, 1991.
- (Mi91b) Microsoft, *Microsoft C/C++ C++ Tutorial* (Versión 7.0), Redmond, WA: Microsoft Corporation, 1991.
- (Pi93) Pittman, M., "Lessons Learned in Managing Object-Oriented Development," *IEEE Software Magazine*, Vol.10, No.1, Enero 1993, pp 43-53.
- (Pr93) Prieto-Diaz, R., "Status Report: Software Reusability," *IEEE Software*, Vol.10, No.3, Mayo 1993, pp. 61-66.
- (Ra92) Ranade, J., y S. Zamir, *C++ Primer for C Programmers*, New York, NY McGraw-Hill, Inc., 1992.
- (Re91) Reed, D.R., "Moving from C to C++," *Object Magazine*, Vol.1, No.3, Septiembre/Octubre, 1991, pp. 46-60.
- (Rl93) Rettig, M., G. Simmons, y J. Thompson, "Extended Objects," *Communications of the ACM*, Vol.36, No.8, Agosto 1993, pp. 19-24.
- (Sa93) Saks, D., "Inheritance, Parte 2," *The C Users Journal*, Mayo 1993, pp.81-89.
- (Sh91) Shiffman, H., "C++ Object-Oriented Extensions to C," *SunWorld*, Vol.4, No.5, Mayo de 1991, pp.63-70.
- (Sk93) Skelly, C., "Pointer Power in C and C++, Parte 1," *The C Users Journal*, Vol.11, No.2, Febrero 1993, pp. 93-98.
- (Sn93) Snyder, A., "The Essence of Objects: Concepts and Terms," *IEEE Software Magazine*, Vol.10, No.1, Enero 1993, pp. 31-42.
- (St91) Stroustrup, B., *The C++ Programming Language* (Second Edition), Reading, MA: Addison-Wesley Series in Computer Science, 1991.
- (St93) Stroustrup, B., "Why Consider Language Extensions?": Maintaining a Delicate Balance," *C++ Report*, Septiembre 1993, pp. 44-51.
- (Vo93) Voss, G., "Objects and Messages," *Windows Tech Journal*, Febrero de 1993, pp. 15-16.
- (Wi93) Wiebel, M., y S. Halladay, "Using OOP Techniques Instead of *switch* in C++," Vol.10, No.10, *The C Users Journal*, Octubre de 1992, pp. 105-106.
- (Wl93) Wilde, N., P. Matthews, y R. Huitt, "Maintaining Object-Oriented Software" *IEEE Software Magazine*, Vol.10, No.1, Enero 1993, pp. 75-80.
- (Wt93) Wilt, N., "Templates in C++," *The C Users Journal*, Mayo de 1993, pp. 33-51.

Apéndice A *

Sintaxis de C

En la notación sintáctica utilizada, las categorías sintácticas (no terminales) se indican en *cursivas* y las palabras literales y los miembros de conjuntos de caracteres (terminales) en **negritas**. Dos puntos después de una terminal introducen su definición. Las definiciones alternas son enlistadas en líneas por separado, excepto cuando estén anteceditas por las palabras “una de”. Un símbolo opcional es presentado mediante el subíndice “opt”, de tal forma que

{expresión_{opt}}

indica una expresión opcional encerrada entre llaves.

Resumen de sintaxis de lenguaje

A.1 Gramática lexicográfica

A.1.1 Componentes léxicos (tokens)

Componente léxico:

- palabra reservada*
- identificador*
- constante*
- cadena literal*
- operador*
- puntuaciones*

componente léxico de preprocesamiento:

- nombre de encabezado*
- identificador*
- número de preprocesador*
- constante de carácter*
- cadena literal*
- operador*

* Certificación de autorización: Este material ha sido condensado y adaptado partiendo de American National Standard for International —Systems Programming Language— C, ANSI/ISO 9899:1990. Se pueden adquirir copias de esta norma de American National Standards Institute en 11 West 42nd. Street, New York, NY 10036.

puntuaciones
cada carácter distinto a un espacio en blanco que no pueda ser uno de los arriba citados

A.1.2 Palabras reservadas

<i>palabra reservada: una de</i>			
auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

A.1.3 Identificadores

identificador:

- no dígito*
- identificador no dígito*
- identificador dígito*

no dígito: uno de:

_ a b c d e f g h i j k l m
n o p q r s t u v w x y z
A B C D E F G H I J K L M
N O P Q R S T U V W X Y Z

dígito: uno de

0 1 2 3 4 5 6 7 8 9

A.1.4 Constantes

constante:

- constante flotante*
- constante entero*
- constante de enumeración*
- constante de carácter*

constante flotante:

- constante fraccionaria sufijo_{opt} flotante parte_{opt} exponencial*
- secuencia de dígitos sufijo_{opt} flotante parte exponencial*

constante fraccionaria:

- secuencia_{opt} de dígitos . secuencia de dígitos*
- secuencia de dígitos.*

parte exponencial:

- signo_{opt} e secuencia de dígitos*
- signo_{opt} E secuencia_{opt} de dígitos*

signo: uno de

+ -

secuencia de dígitos
dígito
secuencia de dígitos dígito

sufijo flotante: uno de
f l F L

constante entera
constante decimal sufijo_{opt} entero
constante octal sufijo_{opt} entero
constante hexadecimal sufijo_{opt} entero

constante decimal:
dígito no cero
constante decimal dígito

constante octal:
0
constante octal dígito octal

constante hexadecimal:
dígito hexadecimal 0x
dígito hexadecimal 0X
constante hexadecimal dígito hexadecimal

dígito no cero: uno de
1 2 3 4 5 6 7 8 9

dígito octal: uno de
0 1 2 3 4 5 6 7

dígito hexadecimal: uno de
0 1 2 3 4 5 6 7 8 9
a b c d e f
A B C D E F

sufijo entero:
sufijo unsigned sufijo_{opt} long
sufijo long sufijo_{opt} unsigned

sufijo unsigned: uno de
u U

sufijo long: uno de
l L

constante de enumeración:
identificador

constante de carácter:
'secuencia c-char'
'secuencia c-char' L

secuencia c-char:
c-char
secuencia c-char c-char

c-char:
cualquier miembro del conjunto de caracteres fuente, a excepción de la comilla sencilla **'**, la diagonal invertida ****, o el carácter de nueva línea
secuencia de escape

secuencia de escape:
secuencia de escape simple
secuencia de escape octal
secuencia de escape hexadecimal

secuencia de escape simple: una de
\' \" **\?** ****
\a \b \f \n \r \t \v

secuencia de escape octal:
*dígito octal *
*dígito octal dígito octal *
*dígito octal dígito octal dígito octal *

secuencia de escape hexadecimal:
dígito hexadecimal \x
secuencia de escape hexadecimal dígito hexadecimal

A.1.5 Cadenas literales

cadena literal:
"secuencia_{opt} s-char"
"secuencia_{opt} s-char" L

secuencia s-char:
s-char
secuencia s-char s-char

s-char:
cualquier miembro del conjunto de caracteres fuente, a excepción de las dobles comillas **"**, la diagonal invertida **** o el carácter de nueva línea
secuencia de escape

A.1.6 Operadores
operador: uno de

[] () . ->
++ -- & * + - ~ ! sizeof
/ % << >> < > <= >= == != ^ | && ||
? :
= *= /= %= += -= <<= >>= &= ^= |=
, # ##

A.1.7 Puntuaciones
puntuaciones: uno de

[] () { } * , : = ... #

A.1.8 Nombres de encabezados
nombre de encabezado:
<h-char-sequence>
“secuencia q-char”

secuencia h-char:
h-char
secuencia h-char h-char

h-char:
cualquier miembro del conjunto de caracteres fuente, a excepción del carácter de nueva línea, y de >

secuencia q-char:
q-char
secuencia q-char q-char

q-char:
cualquier miembro del conjunto de caracteres fuente, a excepción del carácter de nueva línea, y de "

A.1.9 Números de preprocesador
número de preprocesador:
dígito
dígito .
dígito de número de preprocesador
no dígito de número de preprocesador
signo **E** de número de preprocesador
signo **E** de número de preprocesador
. de número de preprocesador

A.2. Gramática estructural de frases

A.2.1 Expresiones
expresión primaria:
identificador
constante
cadena literal
(expresión)

expresión posfija:
expresión primaria
expresión posfija [expresión]
expresión posfija (lista_{opt} de expresión de argumentos)
expresión posfija identificador .
expresión posfija identificador :>
expresión posfija ++
expresión posfija --

lista de expresión de argumentos:
expresión de asignación
lista de expresión de argumentos , expresión de asignación

expresión unaria:
expresión posfija
expresión unaria ++
expresión unaria --
expresión de operador unario de conversión explícita (cast)
expresión unaria sizeof
sizeof (nombre de tipo)

operador unario: uno de
& * + - ~ !

expresión de conversión explícita (cast):
expresión unaria
expresión de conversión explícita (cast) (nombre de tipo)

expresión multiplicativa:
expresión de conversión explícita (cast)
expresión multiplicativa expresión de conversión explícita (cast) *
expresión multiplicativa expresión de conversión explícita (cast) /
expresión multiplicativa expresión de conversión explícita (cast) %

expresión aditiva:
expresión multiplicativa
expresión aditiva expresión multiplicativa +
expresión aditiva expresión multiplicativa -

expresión de desplazamiento:
expresión aditiva
expresión de desplazamiento expresión aditiva <<
expresión de desplazamiento expresión aditiva >>

expresión relacional:
expresión de desplazamiento
expresión de desplazamiento expresión relacional <
expresión de desplazamiento expresión relacional >

expresión relacional *expresión de desplazamiento* <=
expresión relacional *expresión de desplazamiento* >=

expresión de igualdad:
expresión relacional
expresión de igualdad *expresión relacional* ==
expresión de igualdad *expresión relacional* !=

expresión AND:
expresión de igualdad
expresión de igualdad &

expresión OR exclusivo:
expresión AND
expresión OR exclusivo *expresión AND* ^

expresión OR inclusivo:
expresión OR exclusivo
expresión OR inclusivo *expresión OR exclusivo* |

expresión AND lógica:
expresión OR inclusivo
expresión AND lógica *expresión OR inclusivo* &&

expresión OR lógica:
expresión AND lógica
expresión OR lógica *expresión AND lógica* ||

expresión condicional:
expresión OR lógica
expresión OR lógica *expresión* ? *expresión condicional* :

expresión de asignación:
expresión condicional
expresión de asignación unaria *expresión de asignación operador*

operador de asignación: uno de
= *= /= %= += -= <<= >>= &= ^= |=

expresión:
expresión condicional
expresión *expresión condicional* ,

expresión constante:
expresión condicional

A.2.2 Declaraciones

declaración:
declaración de especificadores de inicialización *lista*_{opt} ;

declaración de especificadores:
declaración de especificadores de clase de almacenamiento *especificadores*_{opt}
declaración de especificadores de tipo *especificadores*_{opt}
*declaración de calificadores de tipo de especificadores*_{opt}

lista de declarador de inicialización:
declarador de inicialización
lista de declarador de inicialización *declarador de inicialización* ,

declarador Init:
declarador
declarador = *inicializador*

especificador de clase de almacenamiento:
typedef
extern
static
auto
register

especificador de tipo:
void
char
short
int
long
float
double
signed
unsigned
especificador de estructuras o uniones
especificador de enumeraciones
nombre typedef

especificador de estructura o unión:
*identificador de estructura o unión*_{opt} {*lista de declaración de estructura*}
identificador de estructura o unión

estructura o unión:
struct
union

lista de declaración de estructura:
declaración de estructura
lista de declaración de estructura *declaración de estructura*

declaración de estructura:
lista de calificadores de especificación *lista de declaración de estructura* ;

lista de calificadores de especificación:
 especificador de especificadores de tipo lista de calificadores_{opt}
 especificador de calificadores de tipo lista de calificadores_{opt}

lista de declarador de estructura:
 declarador de estructura
 lista de declarador de estructura declarador de estructura ,

declarador de estructura:
 declarador
 declarador_{opt} : expresión constante

especificador de enumeraciones:
 identificador_{opt} enum { lista de enumerador }
 identificador enum

lista de enumerador:
 enumerador
 lista de enumerador enumerador ,

enumerador:
 constante de enumeración
 constante de enumeración expresión constante =

calificador de tipo:
 const
 volatile

declarador:
 apuntador_{opt} declarador directo

declarador directo:
 identificador
 (declarador)
 declarador directo [expresión_{opt} constante]
 declarador directo (lista de parámetros de tipo)
 declarador directo (lista_{opt} de identificadores)

apuntador:
 lista_{opt} de calificadores de tipo *
 lista_{opt} de calificadores de tipo * apuntador

lista de calificadores de tipo
 calificadores de tipo
 lista de calificadores de tipo calificador de tipo

lista de tipo de parámetros:
 lista de parámetros
 lista de parámetros , . . .

lista de parámetros:
 declaración de parámetros
 lista de parámetros , declaración de parámetros

declaración de parámetros:
 especificador de declaración declarador
 especificador de declaración declarador abstracto_{opt}

lista de identificadores:
 identificador
 lista de identificadores , identificador

nombre de tipo:
 lista de calificador de especificador declarador_{opt} abstracto

declarador abstracto:
 apuntador
 apuntador_{opt} declarador abstracto directo

declarador abstracto directo:
 (declarador abstracto)
 declarador abstracto directo_{opt} [expresión_{opt} constante]
 declarador abstracto directo_{opt} (lista de tipo de parámetros_{opt})

nombre typedef:
 identificador

inicializador:
 expresión de asignación
 { lista de inicializador }
 { lista de inicializador , }

lista de inicializador:
 inicializador
 lista de inicializador , inicializador

A.2.3 Enunciados

enunciado:
 enunciado etiquetado
 enunciado compuesto
 enunciado de expresión
 enunciado de selección
 enunciado de iteración
 enunciado de salto

enunciado etiquetado:
 identificador : enunciado
 expresión constante case expresión :
 enunciado: default

enunciado compuesto:

```
{ listaopt de declaración listaopt de enunciado }
```

lista de declaración:

```
declaración  
declaración lista de declaración
```

lista de enunciado:

```
enunciado  
enunciado lista de enunciado
```

enunciado de expresión:

```
expresiónopt ;
```

enunciado de selección:

```
enunciado ( expresión ) if  
enunciado ( expresión ) if enunciado else  
enunciado ( expresión ) switch
```

enunciado de iteración:

```
enunciado ( expresión ) while  
enunciado do ( expresión ) while ;  
enunciado (expresiónopt ; expresiónopt ; expresiónopt) for
```

enunciado de salto:

```
identificador goto ;  
continue ;  
break ;  
expresiónopt return ;
```

A.2.4 Definiciones externas

unidad de traducción:

```
declaración externa  
unidad de traducción declaración externa
```

declaración externa:

```
definición de función  
declaración
```

definición de función:

```
declarador de especificadoresopt de declaración enunciado compuesto de listaopt de declaración
```

A.3 Directrices de preprocesador

archivo de preprocesador:

```
grupoopt
```

grupo:

```
parte de grupo  
parte de grupo grupo
```

parte de grupo:

```
componentes léxicosopt de preprocesador nueva línea  
sección if  
línea de control
```

sección if:

```
grupo if gruposopt elif grupoopt línea endif
```

grupo if:

```
# if expresión constante grupoopt nueva línea  
# ifdef identificador grupoopt nueva línea  
# ifndef identificador grupoopt nueva línea
```

grupos elif:

```
grupos elif  
grupos elif grupo elif
```

grupo elif:

```
# elif expresión constante grupoopt nueva línea
```

grupo else:

```
# else grupoopt nueva línea
```

línea endif:

```
# endif nueva línea
```

línea de control:

```
# include componentes léxicos nueva línea  
# define lista de remplazo identificador nueva línea  
# define identificador paréntesis izquierdo (lparen) listaopt identificador ) lista de remplazo  
nueva línea  
# undef identificador nueva línea  
# line componentes léxicos nueva línea  
# error componentes léxicosopt nueva línea  
# pragma componentes léxicosopt nueva línea  
# nueva línea
```

lparen:

```
el carácter de paréntesis izquierdo sin espacio en blanco previo
```

lista de remplazo:

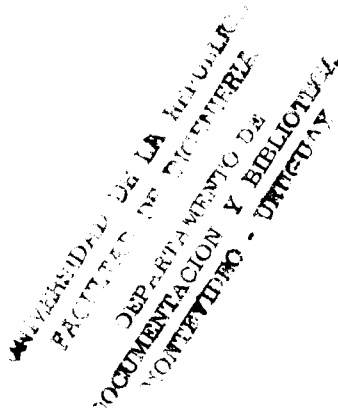
```
componentes léxicosopt
```

componentes léxicos:

```
componente léxico de preprocesador  
componentes léxicos de preprocesador componente léxico
```

nueva línea:

```
el carácter de nueva línea
```



Apéndice B*

Biblioteca estándar

B.1 Errores <errno.h>

EDOM

ERANGE

Estas se expanden a expresiones constantes integrales con valores precisos no cero, utilizables para uso en directrices de preprocesador `#if`.

errno

Un valor de tipo `int` que es definido por varias funciones de biblioteca como un número positivo de error. Al arranque del programa el valor `errno` es cero, pero jamás es definido como cero por ninguna función de biblioteca. Un programa que utilice `errno` para verificación de errores debe definirlo a cero antes de una llamada a una función de biblioteca, y leerlo antes de una subsiguiente llamada a una función de biblioteca. Una función de biblioteca puede guardar el valor de `errno` al entrar y después definirlo como cero, siempre y cuando el valor original sea restaurado, si justo antes del regreso, el valor de `errno` siga siendo cero. El valor de `errno` puede ser definido por una llamada de función de biblioteca como un valor no cero, exista o no exista error, siempre que el uso de `errno` no haya sido documentado en la descripción de función en el estándar.

B.2 Definiciones comunes <stddef.h>

NULL

Una constante de apuntador nula definida por la puesta en práctica.

offsetof (*tipo, designador de miembro*)

Se expande a una expresión constante íntegra de tipo `size_t`, el valor de la cual es el desplazamiento en bytes al miembro de estructura (especificado por el *designador de miembro*) a partir del principio de su tipo de estructura (designado por *tipo*). El *designador de miembro* debe ser de tal forma que, dado

```
static tipo t;
```

* Reconocimiento de autorización: Este material ha sido condensado y adaptado a partir del American National Standard for Information Systems — Programming Language — C, ANSI/ISO 9899:1990. Se pueden adquirir copias de esta norma de la American National Standard Institute en 11 West 42nd Street, New York, NY 10036.

entonces, la expresión `& (t. designador de miembro)` se evalúa a una constante de dirección. (Si el miembro especificado es un campo de bit, el comportamiento queda indefinido).

ptrdiff_t

El tipo entero con signo del resultado de la sustracción de dos apuntadores.

size_t

El tipo entero sin signo del resultado del operador `sizeof`.

wchar_t

Un tipo entero cuyo rango de valores puede representar códigos diferentes para todos los miembros del conjunto extendido de caracteres especificado entre los escenarios soportados; el carácter nulo tendrá el valor de código cero y cada miembro del conjunto básico de caracteres tendrá un valor de código igual a su valor cuando se use como un carácter único en una constante de caracteres íntegra.

B.3 Diagnósticos <assert.h>

void assert(int expression);

La macro **assert** efectúa diagnósticos dentro de programas. Cuando se ejecuta, si **expression** es falsa, la macro **assert** escribe información relativa a la llamada particular que falló (incluyendo el texto del argumento, el nombre del archivo fuente, y el número de línea fuente —éstos últimos son respectivamente los valores de los macros de preprocesador `__FILE__` y `__LINE__`) en el archivo de error estándar en formato definido por la puesta en práctica. El mensaje escrito pudiera aparecer de la forma

```
Assertion failed: expression, file xyz, line nnn
```

La macro **assert** a continuación llama a la función **abort**. Si la directiva de preprocesador

```
#define NDEBUG
```

aparece en el archivo fuente donde **assert.h** esté incluido, cualquier verificación sobre el archivo será ignorada.

B.4 Manejo de caracteres <ctype.h>

Las funciones en esta sección devuelven no cero (verdadero) si, y sólo si, el valor del argumento `c` está conforme con el incluido en la descripción de la función.

int isalnum(int c);

Prueba buscando cualquier carácter para el cual **isalpha** o **isdigit** es verdadero.

int isalpha(int c);

Prueba buscando cualquier carácter para el cual **isupper** o **islower** es verdadero.

int iscntrl(int c);

Prueba la existencia de cualquier carácter de control.

int isdigit(int c);

Prueba buscando cualquier carácter de dígito decimal.

int isgraph(int c);

Busca cualquier carácter de impresión, excepto el espacio (' ').

int islower(int c);

Busca cualquier carácter que sea una letra minúscula.

int isprint(int c);

Busca cualquier carácter de impresión incluyendo el espacio (' ').

int ispunct(int c);

Prueba buscando cualquier carácter de impresión que no sea ni espacio (' ') ni ningún carácter para el cual `isalnum` resulte verdadero.

```
int isspace(int c);
```

Prueba buscando cualquier carácter que sea un carácter estándar de espacio en blanco. Los caracteres estándar de espacio en blanco son: espacio (' '), alimentación de forma ('\f'), nueva línea ('\n'), retorno de carro ('\r'), tabulador horizontal ('\t') y tabulador vertical ('\v').

```
int isupper(int c);
```

Busca cualquier carácter que esté en mayúsculas.

```
int isxdigit(int c);
```

Busca cualquier carácter en dígitos hexadecimales.

```
int tolower(int c);
```

Convierte una letra mayúscula en la letra minúscula correspondiente. Si el argumento es un carácter para el cual `isupper` resulta verdadero y existe un carácter correspondiente para el cual `islower` resulta verdadero, la función `tolower` devuelve el carácter correspondiente; de lo contrario, el argumento se conserva sin modificación.

```
int toupper(int c);
```

Convierte una letra minúscula a la letra mayúscula correspondiente. Si el argumento a un carácter para el cual `islower` resulta verdadero y existe un carácter correspondiente para el cual `isupper` es verdadero, la función `toupper` devuelve el carácter correspondiente; de lo contrario, el argumento se regresa sin modificación.

B.5 Localización <locale.h>

```
LC_ALL
LC_COLLATE
LC_CTYPE
LC_MONETARY
LC_NUMERIC
LC_TIME
```

Estas se expanden a expresiones constantes íntegras con valores precisos, utilizables como primer argumento de la función `setlocale`.

```
NULL
```

Una constante de apuntador nula definida por la puesta en práctica.

```
struct lconv
```

Contiene miembros relacionados con el formato de valores numéricos. La estructura deberá contener por lo menos los siguientes miembros en cualquier orden. En el escenario "C" los miembros tendrán los valores especificados en los comentarios.

```
char p_cs_precedes; /* CHAR_MAX */
char p_sep_by_space; /* CHAR_MAX */
char n_cs_precedes; /* CHAR_MAX */
char n_sep_by_space; /* CHAR_MAX */
char p_sign_posn; /* CHAR_MAX */
char n_sign_posn; /* CHAR_MAX */
```

`char *setlocale(int category, const char *locale);`

La función `setlocale` selecciona la porción apropiada del escenario del programa, tal y como se especifica por los argumentos `category` y `locale`. La función `setlocale` puede ser utilizada para modificar o consultar el escenario actual total del programa o partes del mismo. El valor `LC_ALL` para `category` nombra todo el escenario del programa: los otros valores para `category` nombran sólo una porción del escenario del programa. `LC_COLLATE` afecta el comportamiento de las funciones `strcoll` y `strxfrm`. `LC_CTYPE` afecta el comportamiento de las funciones de manejo de caracteres y de las funciones multibyte. `LC_MONETARY` afecta la información de formato de moneda devuelta por la función `localeconv`. `LC_NUMERIC` afecta el carácter de punto decimal para las funciones de entrada/salida con formato, para las funciones de conversión de cadenas, y para la información de formato no monetario devuelta por `localeconv`. `LC_TIME` afecta el comportamiento de `strftime`.

Un valor de "C" para el `locale` especifica el entorno mínimo para traducción en C; un valor de "" para `locale` especifica el entorno nativo definido por la puesta en práctica. Se pueden pasar a `setlocale` otras cadenas definidas por la puesta en práctica. Al arranque del programa, se ejecuta el equivalente de

```
setlocale(LC_ALL, "C");
```

Si un apuntador a una cadena se da para `locale` y dicha selección puede ser aceptada, la función `setlocale` devuelve un apuntador a la cadena asociada con la `category` especificada para el nuevo escenario. Si la selección no puede ser aceptada, la función `setlocale` devuelve un apuntador nulo y el escenario del programa no es modificado.

Un apuntador nulo para `locale` hace que el función `setlocale` devuelva un apuntador a la cadena asociada con la `category` correspondiente al escenario actual del programa; el escenario del programa no se modifica.

El apuntador a la cadena devuelta por la función `setlocale` es tal que una llamada subsecuente a ese valor de cadena y a su categoría asociada restaurará dicha parte de la escenario del programa. La cadena a la cual se señala será modificada por el programa, pero podría ser sobrescrita por una llamada subsecuente a la función `setlocale`.

```
struct lconv *localeconv(void);
```

La función `localeconv` define los componentes de un objeto con tipo `struct lconv` con valores apropiados para el formato de cantidades numéricas (de moneda y otras) de acuerdo con las reglas del escenario actual.

Los miembros de la estructura con el tipo `char *` son apuntadores a cadenas, cualquiera de los cuales (a excepción de `decimal_point`) puede señalar a "", para indicar que el valor no está disponible en el escenario actual o tiene una longitud cero. Los miembros con tipo `char` son números no negativos, cualquiera de los cuales puede ser `CHAR_MAX` para indicar que el valor no está disponible en el escenario actual.

Los miembros incluyen lo siguiente:

```
char *decimal_point
```

El carácter de punto decimal utilizado para darle formato a cantidades no monetarias.

```
char *thousands_sep
```

El carácter utilizado para separar grupos de dígitos antes del carácter de punto decimal en cantidades no monetarias con formato.

char *grouping

Una cadena cuyos elementos indican el tamaño de cada grupo de dígitos en cantidades no monetarias con formato.

char *int_curr_symbol

El símbolo de moneda internacional aplicable al escenario actual. Los tres primeros caracteres contienen el símbolo de moneda internacional alfabético de acuerdo con lo especificado en ISO 4217:1987. El cuarto carácter (que precede al carácter nulo) es el carácter utilizado para separar el símbolo internacional de moneda de la cantidad monetaria.

char *currency_symbol

El símbolo local de moneda aplicable al escenario actual.

char *mon_decimal_point

El punto decimal utilizado para darle formato a cantidades monetarias.

char *mon_thousands_sep

El separador para grupos de dígitos antes del punto decimal en cantidades monetarias con formato.

char *mon_grouping

Una cadena cuyos elementos indican el tamaño de cada grupo de dígitos en cantidades monetarias con formato.

char *positive_sign

Una cadena utilizada para indicar una cantidad monetaria con formato con valores no negativos.

char *negative_sign

La cadena que se utiliza para indicar una cantidad monetaria con formato de valor negativo.

char int_frac_digits

El número de dígitos fraccionarios (aquellos que aparecen después del punto decimal) al mostrarse en una cantidad monetaria internacional con formato.

char frac_digits

El número de dígitos fraccionarios (los que aparecen después del punto decimal) a mostrarse en una cantidad monetaria con formato.

char p_cs_precedes

Se define como 1 ó como 0 el `currency_symbol` respectivamente antecederá o seguirá al valor en una cantidad monetaria no negativa con formato.

char p_sep_by_space

Definido como 1 ó como 0 el `currency_symbol` respectivamente está o no está separado mediante un espacio de la cantidad monetaria no negativa con formato.

char n_cs_precedes

Definido a 1 ó como 0 el `currency_symbol` respectivamente antecede o sigue al valor en una cantidad monetaria negativa con formato.

char n_sep_by_space

Definido a 1 ó como 0 el `currency_symbol` respectivamente está o no está separado mediante un espacio del valor en una cantidad monetaria negativa con formato.

char p_sign_posn

Definido a un valor indicado en la posición del `positive_sign` para una cantidad monetaria no negativa con formato.

char n_sign_posn

Definido a un valor indicado en la posición del `negative_sign` en cantidad monetaria negativa con formato.

Los elementos de `grouping` y de `mon_grouping` se interpretan de acuerdo con lo siguiente:

CHAR_MAX No se ejecutará más agrupamiento

0 El elemento precedente debe ser utilizado de forma repetida para el resto de los dígitos.

other El valor entero es el número de dígitos que comprenden el grupo actual. Se examinará el siguiente elemento para determinar el tamaño del siguiente grupo de dígitos antes del grupo actual.

Los valores de `p_sign_posn` y de `n_sign_posn` se interpretan de acuerdo con lo siguiente.

0 Paréntesis encerrando la cantidad y `currency_symbol`.

1 La cadena de signos antecede a la cantidad y `currency_symbol`.

2 La cadena de signos sigue a la cantidad y `currency_symbol`.

3 La cadena de signos precede de inmediato antes de `currency_symbol`.

4 La cadena de signos sigue de inmediato a `currency_symbol`.

La función `localeconv` devuelve un apuntador al objeto relleno. La estructura a la cual se señala mediante el valor de regreso no deberá ser modificado por el programa, pero puede ser sobreescrito mediante una llamada subsecuente a la función `localeconv`. En adición, llamadas a la función `setlocale` con las categorías `LC_ALL`, `LC_MONETARY`, o bien `LC_NUMERIC` pueden sobrescribir el contenido de la estructura.

B.6 Matemáticas <math.h>

HUGE_VAL

Una constante simbólica que representa una expresión positiva `double`.

double acos(double x);

Calcula el valor principal del arco cuyo coseno es `x`. Para argumentos que no estén en el rango $[-1, +1]$ ocurrirá un error de dominio. La función `acos` devuelve el arco del coseno en el rango de $[0, \pi]$ radianes.

double asin(double x);

Calcula el valor principal del arco cuyo seno es `x`. Para argumentos que no estén en el rango $[-1, +1]$ ocurrirá un error de dominio. La función `asin` devuelve el seno arco en el rango de $[-\pi/2, +\pi/2]$ radianes.

double atan(double x);

Calcula el valor principal del arco cuya tangente es `x`. La función `atan` devuelve el arco tangente en el rango de $[-\pi/2, +\pi/2]$ radianes.

double atan2(double y, double x);

La función `atan2` calcula el valor principal del arco tangente `y/x`, utilizando los signos de ambos argumentos para determinar el cuadrante del valor devuelto. Puede ocurrir un error de dominio si ambos argumentos son cero. La función `atan2` devuelve el arco tangente de `y/x`, en el rango $[-\pi, +\pi]$ radianes.

double cos(double x);

Calcula el coseno de `x` (medido en radianes).

double sin(double x);

Calcula el seno de `x` (medido en radianes).

double tan(double x);
Devuelve la tangente de **x** (medida en radianes).

double cosh(double x);
Calcula el coseno hiperbólico de **x**. Puede ocurrir un error de rango si la magnitud de **x** es demasiado grande.

double sinh(double x);
Calcula el seno hiperbólico de **x**. Ocurre un error de rango si la magnitud de **x** es demasiado grande.

double tanh(double x);
La función **tanh** calcula la tangente hiperbólica de **x**.

double exp(double x);
Calcula la función exponencial de **x**. Ocurre un error de rango si la magnitud de **x** es muy extensa.

double frexp(double value, int *exp);
Divide el número de punto flotante en una fracción normalizada y una potencia entera de 2. Almacena el entero en el objeto **int** apuntado por **exp**. La función **frexp** devuelve el valor **x**, tal que **x** es un **double** con magnitud en el intervalo [1/2, 1] o cero, y **value** es igual a **x** multiplicado por 2 elevado a la potencia ***exp**. Si **value** es igual a cero, ambas partes del resultado son cero.

double ldexp(double x, int exp);
Multiplica un número de punto flotante por una potencia entera de 2. Puede ocurrir un error de rango. La función **ldexp** devuelve el valor de **x** multiplicado por 2 elevado a la potencia **exp**.

double log(double x);
Calcula el logaritmo natural de **x**. Ocurre un error de dominio si el argumento es negativo. Pudiera ocurrir un error de rango si el argumento es cero.

double log10(double x);
Calcula el logaritmo en base diez de **x**. Ocurre un error de dominio si el argumento es negativo. Pudiera ocurrir un error de rango si el argumento es cero.

double modf(double value, double *iptr);
Divide el argumento **value** en partes enteras y fraccionarias, cada una de las cuales tiene el mismo signo que el argumento. Almacena en la parte entera como un **double** en el objeto al cual apunta **iptr**. La función **modf** devuelve la parte fraccionaria signada de **value**.

double pow(double x, double y);
Calcula **x** elevado a la potencia **y**. Ocurre un error de dominio si **x** es negativo e **y** no es un valor entero. Ocurre un error de dominio si el resultado no puede ser representado cuando **x** es cero e **y** es menor o igual a cero. Puede ocurrir un error de rango.

double sqrt (double x);
Calcula la raíz cuadrada no negativa de **x**. Ocurre un error de dominio si el argumento es negativo.

double ceil(double x);
Calcula el valor integral más pequeño no menor que **x**.

double fabs(double x);
Calcula el valor absoluto del número de punto flotante **x**.

double floor(double x);
Calcula el valor integral más grande no mayor que **x**.

double fmod(double x, double y);
Calcula el residuo en punto flotante de **x** dividido entre **y**.

B.7 Saltos no locales <setjmp.h>

jmp_buf

Un tipo de arreglo adecuado para contener la información necesaria para restaurar un entorno llamador.

int setjmp(jmp_buf env);

Guarda su entorno llamador en el argumento **jmp_buf** para uso posterior por la función **longjmp**.

Si el regreso es debido a una invocación directa, la macro **setjmp** devuelve el valor cero. Si el regreso es debido a una llamada de la función **longjmp**, la macro **setjmp** devuelve un valor no cero. Una invocación a la macro **setjmp** debe de aparecer sólo en alguno de los siguientes contextos:

- en la totalidad de la expresión de control de un enunciado de selección o de iteración;
- un operando de un operador relacional o de igualdad siendo el otro operando una expresión constante entera, con la expresión resultante siendo la expresión de control total de un enunciado de selección o de iteración;
- el operando del operador unario **!** con la expresión resultante siendo la expresión de control total de un enunciado de selección o de iteración; o bien
- la expresión total de un enunciado de expresión.

void longjmp(jmp_buf env, int val);

Devuelve el entorno guardado por la invocación más reciente de la macro **setjmp** en la misma invocación del programa, con el argumento correspondiente **jmp_buf**. Si dicha invocación no ha ocurrido o si en el ínterin la función que contiene la invocación de la macro **setjmp** ha terminado su ejecución, el comportamiento queda indefinido.

Todos los objetos accesibles tienen valores correspondientes al momento en que fue llamado **longjmp**, excepto que los valores de objeto de persistencia automática, que son locales a la función que contiene la invocación de la macro **setjmp** correspondiente que no son del tipo volátil y que han sido modificados entre la invocación **setjmp** y la invocación **longjmp**, quedan indeterminados.

Como pasa por alto los mecanismos usuales de llamada de función y de regreso, **longjmp** ejecutará de forma correcta en el contexto de interrupciones, señales y cualquiera de sus funciones asociadas. Sin embargo, si la función **longjmp** es invocada a partir de un manejador de señales anidado (esto es, desde una función invocada como resultado de una señal iniciada durante el manejo de otra señal), el comportamiento queda indefinido.

Después de que **longjmp** se haya completado, la ejecución del programa continuará como si la invocación correspondiente de la macro **setjmp** hubiera devuelto el valor especificado por **val**. La función **longjmp** no puede hacer que la macro **setjmp** devuelva el valor 0; si **val** es 0, la macro **setjmp** devuelve el valor 1.

B.8 Manejo de señales <signal.h>

sig_atomic_t

El tipo entero de un objeto al cual se puede tener acceso como entidad atómica, inclusive en presencia de interrupciones asincrónicas.

SIG_DFL

SIG_ERR

SIG_IGN

Estas se expanden a expresiones constantes con valores precisos que tienen tipos compatibles con el segundo argumento y con el valor de regreso a la función **signal**, y cuyos valores se comparan en forma desigual con la dirección de cualquier función declarable; en lo que sigue, cada uno de los cuales se expande a una expresión constante entera positiva que es el número de señal para la condición específica:

SIGABRT	terminación anormal, como la que es iniciada por la función abort
SIGFPE	una operación aritmética errónea, como división por cero o una operación que resulte en desbordamiento
SIGILL	detección de una imagen de función inválida, como sería una instrucción ilegal
SIGINT	recepción de una señal de atención interactiva
SIGSEGV	un acceso inválido a almacenamiento
SIGTERM	una solicitud de terminación enviada al programa.

Una puesta en práctica no necesita generar ninguna de estas señales, excepto como resultado de llamadas explícitas a la función **raise**.

```
void (*signal(int sig, void (*func)(int)))(int);
```

Escoge una de tres formas en la cual será subsecuentemente manejado el número de señal **sig**. Si el valor de **func** es **SIG_DEF**, ocurrirá el manejo por omisión de la señal. Si el valor de **func** es **SIG_IGN**, la señal será ignorada. De lo contrario, cuando dicha señal ocurra **func** apuntará a la función a llamarse. Dicha función es un *manejador de señal*.

Cuando ocurre una señal, si **func** apunta a una función, primero se ejecuta el equivalente de **signal(sig, SIG_DFL)** o se ejecuta un bloqueo de la señal, definido por la puesta en práctica. (Si el valor de **sig** es **SIGILL**, la restauración de **SIG_DFL** dependerá de la puesta en práctica). A continuación se ejecuta el equivalente de **(*func)(sig)** La función **func** puede terminar ejecutando un enunciado **return** o llamando a la función **abort**, **exit** o **longjmp**. Si **func** ejecuta un enunciado **return** y el valor de **sig** era **SIGFPE** o cualquier otro valor definido por la puesta en práctica correspondiente a una excepción de cómputo, el comportamiento queda indefinido. De lo contrario, el programa continuará en su ejecución desde el punto en que fue interrumpido.

Si la señal ocurre de forma distinta que como resultado a la llamada de la función **abort** o **raise**, el comportamiento queda indefinido si el manejador de señal llama a cualquier función de la biblioteca estándar distinta a la función **signal** misma (con un primer argumento del número de señal correspondiente a la señal que causó la invocación del manejador) o se refiere a cualquier otro objeto con persistencia estática, distinto que mediante la asignación de un valor a una variable de persistencia estática del tipo **volatile sig_atomic_t**. Además, si dicha llamada a una función **signal** resulta en un regreso **SIG_ERR**, el valor de **errno** queda indeterminado.

Al arranque del programa, el equivalente de

```
signal(sig, SIG_IGN);
```

puede ser ejecutado para algunas señales seleccionadas de una forma definida por la puesta en práctica; el equivalente de

```
signal(sig, SIG_DFL);
```

es ejecutado para todas las demás señales definidas por la puesta en práctica. Si la solicitud puede ser aceptada, la función **signal** devuelve el valor de **func** para la llamada más reciente a **signal** para la señal especificada **sig**. De lo contrario, se devuelve un valor de **SIG_ERR** y en **errno** queda almacenado un valor positivo.

```
int raise(int sig);
```

La función **raise** envía la señal **sig** al programa en ejecución. Si tiene éxito la función **raise** devuelve cero, y no cero si no lo tiene.

B.9 Argumentos variables <stdarg.h>

```
va_list
```

Un tipo adecuado para contener información necesaria para las macros **va_start**, **va_arg**, y **va_end**. Si se desea tener acceso a los distintos argumentos, la función llamada declarará un objeto

(llamada en esta sección **ap**) con el tipo **va_list**. El objeto **ap** puede ser pasado como argumento a otra función; si dicha función invoca la macro **va_arg** con el parámetro **ap**, el valor de **ap** en la función llamada es determinada y será pasada a la macro **va_end** antes de cualquier otra referencia **ap**.

```
void va_start(va_list ap, parmN);
```

Será invocada antes de cualquier acceso a argumentos sin nombre. La macro **va_start** inicializa **apuntador** para uso subsecuente por las macros **va_arg** y **va_end**. El parámetro *parmN* es el identificador del parámetro más a la derecha de la lista de parámetros variables en la definición de función (una justo antes de la coma , ...). Si se declara el parámetro *parmN* con la clase de almacenamiento **register**, con un tipo de función o de arreglo, o con un tipo que no sea compatible con el tipo que resulte después de la aplicación de las promociones de argumento por omisión, el comportamiento quedará indefinido.

```
tipo va_arg(va_list, tipo);
```

Se expande a una expresión que tiene el tipo y el valor del siguiente argumento dentro de la llamada. El parámetro **ap** será el mismo que el de **va_list ap** inicializado por **va_start**. Cada invocación de **va_arg** modifica **ap** de tal forma, que los valores de argumentos sucesivos sean regresados en orden. El parámetro *type* es el nombre de tipo definido de tal forma, que el tipo de un apuntador a un objeto que tiene el tipo especificado se pueda obtener mediante la colocación postfija de un * a *type*. Si no existe un argumento siguiente o si *type* no es compatible con el tipo del siguiente argumento (como promovido de acuerdo con las promociones de argumento por omisión), el comportamiento queda indefinido. La primera invocación de la macro **va_arg** después de la macro **va_start** devuelve el valor del argumento después del especificado por *parmN*. Invocaciones subsiguientes devuelven en sucesión los valores de los argumentos siguientes.

```
void va_end(va_list ap);
```

Facilita el regreso normal de una función cuya lista de argumentos variables fue referida mediante la expansión de **va_start** que inicializó **va_list ap**. La macro **va_end** puede modificar de tal forma **ap**, que ya no sea utilizable (sin una invocación intermedia de **va_start**). Si no existe una invocación correspondiente de la macro **va_start** o si la macro **va_end** no se invoca antes del regreso, el comportamiento queda indefinido.

B.10 Entrada/salida <stdio.h>

```
_IOBF
_IOLBF
_IONBF
```

Expresiones constantes enteras con valores distintos, adecuadas para uso como el tercer argumento a la función **setvbuf**.

```
BUFSIZ
```

Una expresión constante entera, que representa el tamaño del búfer utilizado por la función **setvbuf**.

```
EOF
```

Una expresión constante entera negativa, devuelta por varias funciones a fin de indicar fin de archivo, esto es, no más entradas a partir de un flujo.

```
FILE
```

Un tipo de objeto capaz de registrar toda la información necesaria para controlar un flujo, incluye el indicador de posición de archivo, un apuntador a su búfer asociado (si es que existe alguno), un indicador de error que registre si ha ocurrido algún error de escritura/lectura y un indicador de fin de archivo que registra si se ha llegado al fin del archivo.

FILENAME_MAX
Una expresión constante entera que tiene el tamaño necesario para un arreglo de **char** lo suficiente extensa para contener la cadena de nombre de archivo más larga posible que la puesta en práctica garantiza es posible abrir.

FOPEN_MAX
Una expresión constante entera que es el número mínimo de archivos que la puesta en práctica garantiza se abrirán en forma simultánea.

fpos_t
Un tipo de objeto capaz de registrar toda la información necesaria para especificar todas las posiciones únicas dentro de un archivo.

L_tmpnam
Una expresión constante integral que es del tamaño necesario para un arreglo **char** lo suficiente extensa para contener una cadena de nombre de archivo temporal generado por una función **tmpnam**.

NULL
Una constante de apuntador nulo definido por la puesta en práctica.

SEEK_CUR
SEEK_END
SEEK_SET
Expresiones constantes integrales con valores precisos, adecuadas para uso como el tercer argumento de la función **fseek**.

size_t
El tipo entero sin signo del resultado del operador **sizeof**.

stderr
Expresión del tipo “apuntador a **FILE**” que señala al objeto **FILE** asociado con el flujo estándar de error.

stdin
Expresión del tipo “apuntador a **FILE**” que señala al objeto **FILE** asociado con el flujo de entrada estándar.

stdout
Expresión del tipo “apuntador a **FILE**” que señala al objeto **FILE** asociado con el flujo de salida estándar.

TMP_MAX
Una expresión constante entera que es el número mínimo de nombres únicos de archivo que se generarán mediante la función **tmpnam**. El valor de la macro **TMP_MAX** será de por lo menos 25.

int remove(const char *filename);
Hace que el archivo cuyo nombre es la cadena a la cual señala **filename** ya no sea accesible para dicho nombre. Cualquier intento subsiguiente para abrir dicho archivo utilizando este nombre fallará, a menos que sea creado de nuevo. Si el archivo está abierto, el comportamiento de la función **remove** queda definido por la puesta en práctica. La función **remove** devuelve cero si la operación tiene éxito, y no cero si falla.

int rename(const char *old, const char *new);
Hace que el archivo cuyo nombre es la cadena a la cual señala **old** que de ahí en adelante se conozca por el nombre dado por la cadena a la cual señala **new**. El archivo de nombre **old** ya no es accesible utilizando ese nombre. Si antes de la llamada a la función **rename** existe un archivo llamado mediante la cadena a la cual señala **new**, el comportamiento queda definido por la puesta en práctica. La función

rename devuelve cero si la operación tiene éxito, no cero si falla, en cuyo caso si el archivo ya existía aún se llamará con su nombre original.

FILE *tmpfile(void);
Genera un archivo temporal binario que será eliminado de forma automática cuando se cierre o al final del programa. Si el programa termina en forma anormal, si este programa temporal abierta se elimina o no, depende de la puesta en práctica. El archivo se abre para actualización mediante el modo “**wb+**”. La función **tmpfile** devuelve un apuntador al flujo del archivo que ha sido creado. Si el archivo no puede ser creado, la función **tmpfile** devuelve un apuntador nulo.

char *tmpnam(char *s)
La función **tmpnam** genera una cadena que es un nombre válido de archivo y que no es el mismo que el nombre de un archivo existente. La función **tmpnam** genera una cadena distinta cada vez que es llamada, hasta **TMP_MAX** veces. Si es llamada más de **TMP_MAX** veces, el comportamiento queda definido por la puesta en práctica.
Si el argumento es un apuntador nulo, la función **tmpnam** deja su resultado en un objeto estático interno y devuelve un apuntador a dicho objeto. Llamadas subsecuentes a la función **tmpnam** pueden modificar el mismo objeto. Si el argumento no es un apuntador nulo, se supone que debe señalar a un arreglo de por lo menos **L_tmpnam char**; la función **tmpnam** escribe su resultado en dicho arreglo y devuelve el argumento como su valor.

int fclose(FILE *stream);
La función **fclose** hace que el flujo al cual señala **stream** sea vaciado y el archivo asociado se cierre. Cualquier dato en archivo temporal o búfer aún no escrito para el flujo es entregado al entorno huésped para ser escrito al archivo; cualesquiera datos en archivos temporales no leídos serán descartados. El flujo queda desasociado del archivo. Si el búfer asociado fue asignado en forma automática, queda desasignado. La función **fclose** devuelve cero si el flujo fue cerrado de forma exitosa o bien **EOF** si se detectaron algunos errores.

int fflush(FILE *stream);
Si **stream** señala a un flujo de salida o a uno actualizado en el cual la operación más reciente no ha sido introducida, la función **fflush** hace que todos los datos aún no escritos para dicho flujo sean entregados al entorno huésped o se escriban al archivo; de lo contrario, el comportamiento queda indefinido.
Si **stream** es un apuntador nulo, la función **fflush** lleva a cabo su acción de vaciado sobre todos los flujo para los cuales el comportamiento se ha definido arriba. Si ocurre algún error de escritura la función **fflush** devuelve **EOF**, de lo contrario devuelve cero.

FILE *fopen(const char *filename, const char *mode);
La función **fopen** abre el archivo cuyo nombre es la cadena a la cual señala **filename** y asocia un flujo al mismo. El argumento **mode** señala una cadena que empieza con alguna de las secuencias siguientes:

r	abre archivo de texto para lectura.
w	trunca a cero longitud o crea un archivo de texto para escritura.
a	agrega; abre o crea un archivo de texto para escritura al fin de archivo.
rb	abre archivo binario para lectura.
wb	trunca a longitud cero o crea archivo binario para escritura.
ab	agrega; abre o crea un archivo binario para escritura al fin de archivo.
r+	abre archivo de texto para actualizar (lectura y escritura).
w+	trunca a longitud cero o crea un archivo de texto para actualizar.

a+	agrega; abre o crea un archivo de texto para actualizar, escribiendo al fin de archivo.
r+b o rb+	abre archivo binario para actualizar (lectura y escritura).
w+b o wb+	trunca a longitud cero o crea un archivo binario para actualizar.
a+b o ab+	agrega; abre o crea un archivo binario para actualizar, escribiendo al fin de archivo.

Abrir un archivo en modo de lectura ('r' como el primer carácter en el argumento `mode`) fallará si el archivo no existe o no puede ser leído. Abrir el archivo con el modo de agregar ('a' como el primer carácter en el argumento `mode`) hace que todas las escrituras subsiguientes al archivo se obliguen a lo que en ese momento sea el fin del archivo, independiente de llamadas a la función `fseek`. En algunas puestas en práctica, la apertura de un archivo binario en modo de agregar ('b' como segundo y tercer caracteres en la lista anterior de los valores de argumento `mode`) puede colocar en forma inicial el indicador de posición de archivo para el flujo más allá de los últimos datos escritos, debido al relleno de carácter nulo.

Cuando se abre un archivo con el modo de actualizar ('+' como segundo y tercer carácter en la lista anterior de valores de argumento `mode`), se puede llevar a cabo tanto entrada como salida sobre el flujo asociado. Sin embargo, la salida no puede ser seguida en forma directa por entrada, sin una llamada intermedia a la función `fflush` o a la función de posicionamiento de archivo (`fseek`, `fsetpos` o `rewind`), y la entrada no puede ser de manera directa seguida por salida sin una llamada intermedia a una función de posicionamiento de archivo, a menos de que la operación de entrada se encuentre con fin de archivo. La apertura (o creación) de un archivo de texto en el modo de actualizar, puede quizás abrir (o crear) un flujo binario en algunas puestas en práctica particulares.

Una vez abierto, un flujo está por completo equipado con memoria temporal (búfer) si y sólo si se puede determinar que no se refiere a un dispositivo interactivo. Los indicadores de error y de fin de archivo para el flujo estarán desactivados. La función `fopen` devuelve un apuntador al objeto que controla el flujo. Si la operación de apertura falla, `fopen` devuelve un apuntador nulo.

```
FILE *freopen(const char *filename, const char *mode,
              FILE *stream);
```

La función `freopen` abre el archivo cuyo nombre es la cadena a la cual señala `filename` y asocia el flujo al cual apunta `stream`. El argumento `mode` se utiliza de la misma forma que en la función `fopen`.

La función `freopen` intenta primero cerrar cualquier archivo que esté asociado con el flujo especificado. Se ignorará cualquier falla para cerrar el archivo con éxito. Los indicadores de error y de fin de archivo serán desactivados. Si falla la operación de apertura la función `freopen` devuelve un apuntador nulo. De lo contrario, `freopen` devuelve el valor de `stream`.

```
void setbuf(FILE *stream, char *buf);
```

La función `setbuf` es equivalente a la función `setvbuf` que se invoca con los valores `_IOFBF` para `mode` y `BUFSIZ` para `size`, o (buf es un apuntador nulo) con el valor `_IONBF` para `mode`. La función `setbuf` no devuelve valor.

```
int setvbuf(FILE *stream, char *buf, int mode, size_t size);
```

La función `setvbuf` puede ser utilizada sólo cuando el flujo al cual señala `stream` ha sido asociado con un archivo abierto y antes de cualquier otra operación se haya ejecutado sobre el flujo. El argumento `mode` determina como se harán los búfers de `stream`, como sigue: `_IOFBF` hará que existan búfers completos para entradas/salidas; `_IOLBF` creará búfer de línea para entrada/salida; `_IONBF` hará que las entradas y salidas no tengan búfer. Si `buf` no es un apuntador nulo, el arreglo al cual señala podrá ser utilizado en vez de un búfer asignado por la función `setvbuf`. El argumento `size` especifica el tamaño del arreglo. Los contenidos del arreglo en cualquier momento quedan indeterminados. La función `setvbuf` devuelve cero si tiene éxito o algún valor no cero si se ha dado un valor inválido para `mode` o si la solicitud no puede ser aceptada.

```
int fprintf(FILE *stream, const char *format, ...);
```

La función `fprintf` escribe salida al flujo al cual señala `stream`, bajo control de la cadena al cual señala `format` que define como los argumentos subsecuentes son convertidos para la salida. Si no hay suficientes argumentos para el formato, el comportamiento queda indefinido. Si el formato se termina aunque queden argumentos, los argumentos en exceso se evalúan (como siempre), pero serán ignorados. La función `fprintf` devuelve cuando se encuentra con el fin de la cadena de formato. Vea el capítulo 9, "Entradas/salidas con formato", para una descripción detallada de las especificaciones de conversión de salida. La función `fprintf` devuelve el número de caracteres transmitidos, o un valor negativo si ha ocurrido un error de salida.

```
int fscanf(FILE *stream, const char *format, ...);
```

La función `fscanf` lee la entrada desde el flujo al cual señala `stream`, bajo control de la cadena a la cual apunta `format` que define las secuencias de entrada admisibles y cómo deben de ser convertidas para asignación, utilizando los argumentos subsecuentes como apuntadores a los objetos que han de recibir la entrada convertida. Si para el formato existen argumentos insuficientes, el comportamiento queda indefinido. Si el formato se termina aunque sobren argumentos, los argumentos excedentes se evalúan (como siempre), pero por lo demás se ignorarán. Vea el capítulo 9, "Entradas/salidas con formato", para una descripción detallada de las especificaciones de conversión de entrada.

La función `fscanf` devuelve el valor de la macro `EOF` si ocurre una falla de entrada antes de cualquier conversión. De lo contrario, la función `fscanf` devuelve el número de elementos de entrada asignados, mismos que pueden resultar menos de los proveídos inclusive cero, en caso de una falla temprana de coincidencias.

```
int printf(const char *format, ...);
```

La función `printf` es equivalente a `fprintf` con la interposición del argumento `stdout` antes de los argumentos a `printf`. La función `printf` devuelve el número de caracteres transmitidos o un valor negativo si ocurrió un error de salida.

```
int scanf(const char *format, ...);
```

La función `scanf` es equivalente a `fscanf` con la interposición del argumento `stdin` antes de los argumentos a `scanf`. Si ocurre una falla de entrada antes de cualquier conversión la función `scanf` devuelve el valor de la macro `EOF`. De lo contrario, `scanf` devuelve el número de elementos de entrada asignados, mismos que pueden ser menos de los proveídos o inclusive cero, en el caso de una falla temprana de coincidencia.

```
int sprintf(char *s, const char *format, ...);
```

La función `sprintf` es equivalente a `fprintf`, excepto que el argumento `s` especifica un arreglo al cual deberá de ser escrita la salida generada, en vez de a un flujo. Se escribe un carácter nulo al final de los caracteres escritos; no se cuenta como una parte de la suma devuelta. El comportamiento de copia entre objetos que se superponen queda indefinido. La función `sprintf` devuelve el número de caracteres escritos por el arreglo, sin contar el carácter nulo de terminación.

```
int sscanf(const char *s, const char *format, ...);
```

La función `sscanf` es equivalente a `fscanf`, excepto que el argumento `s` especifica una cadena a partir de la cual se obtendrá la entrada, más bien que a partir de un flujo. El llegar al final de la cadena es equivalente a encontrar fin de archivo para la función `fscanf`. Si la copia ocurre entre objetos que se superponen, el comportamiento queda indefinido.

La función `sscanf` devuelve el valor de la macro `EOF` si ocurre una falla de entrada antes de cualquier conversión. Lo contrario, la función `sscanf` devuelve el número de elementos de entrada asignados, mismos que pueden ser menos de los proveídos inclusive cero, en el caso de una falla temprana de coincidencia.

```
int vfprintf(FILE *stream, const char *format, va_list arg);
```

La función `vfprintf` es equivalente a `fprintf`, con la lista de argumentos variables remplazados por `arg`, que se inicializa mediante la macro `va_start` (y mediante posibles llamadas subsecuentes

va_arg). La función **vfprintf** no invoca a la macro **va_end**. La función **vfprintf** devuelve el número de caracteres transmitidos o un valor negativo si ocurrió un error de salida.

int vprintf(const char *format, va_list arg);

La función **vprintf** es equivalente a **printf**, con la lista de argumentos variables remplazadas por **arg**, que debe haber sido inicializada por la macro **va_start** (y llamadas subsecuentes posibles **va_arg**). La función **vprintf** no invoca a la macro **va_end**. La función **vprintf** devuelve el número de caracteres transmitidos o un valor negativo si ocurrió un error de salida.

int vsprintf(char *s, const char *format, va_list arg);

La función **vsprintf** es equivalente a **sprintf**, con la lista de argumentos variables remplazadas por **arg**, misma que deberá haber sido inicializada por la macro **va_start** (y subsecuentes posibles llamadas **va_arg**). La función **vsprintf** no invoca a la macro **va_end**. Si la copia se lleva a cabo entre objetos que se superponen, el comportamiento queda indefinido. La función **vsprintf** devuelve el número de caracteres escritos en el arreglo, sin contar el carácter nulo de terminación.

int fgetc(FILE *stream);

La función **fgetc** obtiene el carácter siguiente (si está presente) como un **unsigned char** convertido a un **int**, del flujo de entrada al cual señala **stream** y avanza el indicador de posición de archivo asociado correspondiente al flujo (si está definido). La función **fgetc** devuelve el siguiente carácter del flujo de entrada al cual señala **stream**. Si el flujo está al fin de archivo, se activa el indicador de fin de archivo para el flujo y **fgetc** devuelve **EOF**. Si ocurre un error de lectura, se activa el indicador de error para el flujo y **fgetc** devuelve **EOF**.

char *fgets(char *s, int n, FILE *stream);

La función **fgets** lee por lo menos uno menos que el número de caracteres definido por **n** del flujo al cual señala **stream** al arreglo al cual señala **s**. No se leen caracteres adicionales después del carácter de nueva línea (mismos que se conserva) o después de fin de archivo. Se escribe un carácter nulo inmediatamente después del último carácter leído en el arreglo.

Si tiene éxito la función **fgets** devuelve **s**. Si se encuentra con un fin de archivo y no se han leído caracteres al arreglo, el contenido del arreglo se conserva sin modificación y se devuelve un apuntador nulo. Si durante la operación ocurre un error de lectura, el contenido del arreglo queda indeterminado y se devuelve un apuntador nulo.

int fputc(int c, FILE *stream);

La función **fputc** escribe el carácter definido por **c** (convertido a **unsigned char**) al flujo de salida al cual apunta **stream**, en la posición indicada por el indicador de posición de archivo asociado para el flujo (si está definido), y adelanta en forma apropiada dicho indicador. Si el archivo no puede aceptar solicitudes de posicionamiento o si el flujo fue abierto en modo de agregar, el carácter será agregado al flujo de salida. La función **fputc** devuelve el carácter escrito. Si ocurre un error de escritura, se activará el indicador de error para el flujo y **fputc** devolverá **EOF**.

int fputs(const char *s, FILE *stream);

La función **fputs** escribe la cadena a la cual señala **s** en el flujo al cual señala **stream**. No es escrito el carácter nulo de terminación. Si ocurre un error de escritura la función **fputs** devuelve **EOF**; de lo contrario devolverá un valor no negativo.

int getc(FILE *stream);

La función **getc** es equivalente a **fgetc**, excepto que cuando es puesta en práctica como una macro, pudiera evaluar más de una vez **stream** —el argumento deberá ser una expresión sin efectos colaterales.

La función **getc** devuelve el carácter siguiente del flujo de entrada al cual señala **stream**. Si el flujo está al fin de archivo, se activará el indicador de fin de archivo para el flujo y **getc** devuelve **EOF**. Si ocurre un error de lectura, el indicador de error para el flujo se activará y **getc** devolverá **EOF**.

int getchar(void);

La función **getchar** es equivalente a **getc**, con el argumento **stdin**. La función **getchar** devuelve el siguiente carácter del flujo de entrada al cual señala **stdin**. Si el flujo está a fin de archivo, se activará el indicador de fin de archivo para el flujo y **getchar** devolverá **EOF**. Si ocurre un error de lectura, se activará el indicador de error para el flujo y **getchar** devolverá **EOF**.

char *gets(char *s);

La función **gets** lee caracteres del flujo de entrada al cual señala **stdin**, al arreglo apuntado por **s**, hasta que se encuentra fin de archivo o se lee un carácter de nueva línea. Cualquier carácter de nueva línea será descartado o si no se escribirá un carácter nulo después del último carácter leído al arreglo. La función **gets** devuelve **s** si tiene éxito. Si se encuentra un fin de archivo y no se han leído caracteres al arreglo, el contenido del arreglo se conservará sin modificación y se devolverá un apuntador nulo. Si durante la operación ocurre un error de lectura, el contenido del arreglo queda indeterminado y se devuelve un apuntador nulo.

int putc(int c, FILE *stream);

La función **putc** es equivalente a **fputc**, excepto que cuando se pone en práctica como una macro, pudiera evaluar **stream** más de una vez, por lo que el argumento no debería nunca ser una expresión con efectos colaterales. La función **putc** devuelve el carácter escrito. Si ocurre un error de escritura, se activa el indicador de error correspondiente al flujo y **putc** devuelve **EOF**.

int putchar(int c);

La función **putchar** es equivalente a **putc** con **stdout** como segundo argumento. La función **putchar** devuelve el carácter escrito. Si ocurre un error de escritura, se activa el indicador de error para el flujo y **putchar** devuelve **EOF**.

int puts(const char *s);

La función **puts** escribe la cadena a la cual señala **s** en el flujo al cual señala **stdout**, y agrega un carácter de nueva línea a la salida. El carácter nulo de terminación no es escrito. La función **puts** devuelve **EOF** si ocurre un error de escritura; de lo contrario, devuelve un valor no negativo.

int ungetc(int c, FILE *stream);

La función **ungetc** coloca el carácter especificado por **c** (convertido a un **unsigned char**) de regreso en el flujo de entrada al cual señala **stream**. Los caracteres devueltos serán regresados en lecturas subsecuentes de dicho flujo en orden inverso a su entrada. Una llamada intermedia exitosa (con el flujo al cual apunta **stream**) a un función de posicionamiento de archivo (**fseek**, **fsetpos**, o **rewind**) descartará todos los caracteres vueltos a colocar para el flujo. El almacenamiento externo correspondiente al flujo se mantiene sin modificación.

Un carácter de los regresados queda garantizado. Si una función **ungetc** es llamada demasiadas veces sobre el mismo flujo sin una operación intermedia de posicionamiento de lectura o de archivo sobre dicho flujo, la operación pudiera fallar. Si el valor de **c** se iguala al de la macro **EOF**, la operación fallará y el flujo de entrada queda sin modificar.

Una llamada exitosa a la función **ungetc** desactiva el indicador de fin de archivo correspondiente al flujo. El valor del indicador de posición de archivo para el flujo después de leer o descartar todos los caracteres introducidos y regresados será el mismo que era antes de que los caracteres fueran retornados. Para un flujo de texto, el valor de este indicador de posición de archivo después de una llamada a la función **ungetc** queda sin especificar en tanto todos los caracteres devueltos al flujo sean leídos o descartados. En el caso de un flujo binario, su indicador de posición de archivo queda determinado mediante cada llamada exitosa a la función **ungetc**; si su valor antes de la llamada era cero, quedará después de la llamada indeterminado. La función **ungetc** devuelve el carácter devuelto al flujo después de la conversión o si no en caso de fallo de la operación **EOF**.

size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);

La función **fread** lee, al arreglo al cual señala **ptr**, hasta **nmemb** elementos cuyos tamaños están definidos por **size**, del flujo al cual señala **stream**. El indicador de posición de archivo para el flujo

(si está definido) se avanza en el número de caracteres con éxito leídos. Si ocurre algún error, queda indeterminado el valor resultante del indicador de posición de archivo para el flujo. Si se lee un elemento en forma parcial, su valor también queda indeterminado.

La función `fread` devuelve el número de elementos leídos, mismos que pudieran ser menos de `nmemb` si se encuentra con un error de lectura o con el fin de archivo. Si `size` o `nmemb` es cero, `fread` devuelve cero y tanto el contenido del arreglo como el estado del flujo se conservan sin modificar.

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
```

La función `fwrite` escribe, desde el arreglo al cual señala `ptr` hasta `nmemb` elementos cuyo tamaño está definido por `size`, al flujo al cual señala `stream`. El indicador de posición de archivo para el flujo (si está definido) se avanza en el número de carácter escritos con éxito. Si ocurre un error, el valor resultante de la posición de archivo para el flujo queda indeterminado. La función `fwrite` devuelve el número de elementos escritos, mismo que sólo en el caso de que se encuentre un error de escritura será menor que `nmemb`.

```
int fgetpos(FILE *stream, fpos_t *pos);
```

La función `fgetpos` almacena el valor actual del indicador de posición de archivo para el flujo al cual señala `stream` en el objeto al cual apunta `pos`. El valor almacenado contiene información no especificada utilizable por la función `fsetpos` para recolocar el flujo a su posición en el momento de la llamada de la función `fgetpos`. En caso de éxito, la función `fgetpos` devuelve cero; en caso de falla, la función `fgetpos` devuelve no cero y almacena un valor positivo definido según la puesta en práctica en `errno`.

```
int fseek(FILE *stream, long int offset, int whence);
```

La función `fseek` define el indicador de posición de archivo para el flujo al cual señala `stream`. En el caso de un flujo binario, la nueva posición, medida en caracteres a partir del principio del archivo, se obtiene añadiendo `offset` a la posición especificada por `whence`. La posición especificada es el principio del archivo en el caso de que `whence` sea `SEEK_SET`, el valor actual del indicador de posición de archivo si `whence` es `SEEK_CUR`, o fin de archivo si es `SEEK_END`. Un flujo binario no necesariamente debe aceptar en forma significativa llamadas `fseek` con un valor `whence` de `SEEK_END`. En el caso de un flujo de texto, `offset` deberá de ser cero, o un valor devuelto por una llamada anterior a la función `ftell` sobre el mismo flujo y `whence` deberá ser `SEEK_SET`.

Una llamada con éxito a la función `fseek` desactiva el indicador de fin de archivo para el flujo y deshace cualquier efecto de la función `ungetc` sobre el mismo archivo. Después de una llamada a `fseek`, la siguiente operación en un flujo de actualizar puede ser entrada o salida. La función `fseek` devuelve no cero sólo en el caso de una solicitud que no puede ser satisfecha.

```
int fsetpos(FILE *stream, const fpos_t *pos);
```

La función `fsetpos` define el indicador de posición de archivo para el flujo al cual señala `stream` de acuerdo al valor del objeto al cual señala `pos`, mismo que debe de ser un valor obtenido de una llamada anterior a la función `fgetpos` sobre el mismo flujo. Una llamada con éxito a la función `fsetpos` desactiva el indicador de fin de archivo para el flujo y deshace cualquier efecto de la función `ungetc` del mismo flujo. Después de una llamada a `fsetpos`, la siguiente operación sobre un flujo de actualizar puede ser introducida o extraída. Si tiene éxito, la función `fsetpos` devuelve cero; en caso de falla, la función `fsetpos` devuelve no cero y almacena un valor positivo definido según la puesta en práctica en `errno`.

```
long int ftell(FILE *stream);
```

La función `ftell` obtiene el valor actual del indicador de posición de archivo del flujo al cual señala `stream`. En el caso de un archivo binario, el valor es el número de caracteres a partir del principio del archivo. En el caso de un archivo de texto, el indicador de posición de archivo contiene información no especifica, utilizable por la función `fseek` para devolver el indicador de posición de archivo para el flujo a su posición en el momento de la llamada `ftell`; la diferencia entre estos dos valores devueltos

no es necesariamente una medida significativa del número de caracteres escritos o leídos. En caso de éxito, una función `ftell` devuelve el valor actual del indicador de posición de archivo para el flujo. En caso de falla, la función `ftell` devuelve `-1L` y almacena un valor positivo definido por la puesta en práctica en `errno`.

```
void rewind(FILE *stream);
```

La función `rewind` define el indicador de posición de archivo para el flujo al cual señala `stream` al principio del mismo. Es equivalente a

```
(void) fseek(stream, 0L, SEEK_SET)
```

excepto que el indicador de error del flujo también es desactivado.

```
void clearerr(FILE *stream);
```

La función `clearerr` desactiva los indicadores de fin de archivo y de error para el flujo al cual señala `stream`.

```
int feof(FILE *stream);
```

La función `feof` prueba el indicador de fin de archivo en el flujo al cual señala `stream`. La función `feof` devuelve no cero si y sólo si el indicador de fin de archivo está activo para `stream`.

```
int ferror(FILE *stream);
```

La función `ferror` prueba el indicador de error para el flujo al cual señala `stream`. La función `ferror` devuelve no cero si y sólo si el indicador de error está activo para `stream`.

```
void perror(const char *s);
```

La función `perror` traduce el número de error en la expresión entera `errno` en un mensaje de error. Escribe una secuencia de caracteres al flujo de error estándar, de tal forma que: primero (si `s` no es un apuntador nulo y el carácter al cual señala `s` no es un carácter nulo), escribe la cadena a la cual señala `s` seguida por `(:)` y un espacio; a continuación escribe un mensaje de error apropiado seguido por un carácter de nueva línea. El contenido de las cadenas de mensaje de error son las mismas que devuelve la función `strerror` mediante el argumento `errno`, que están definidas por la puesta en práctica.

B.11 Utilerías generales <stdlib.h>

EXIT_FAILURE

EXIT_SUCCESS

Expresiones enteras que pueden ser utilizadas como argumentos de la función `exit` para devolver estados de terminación con o sin éxito, al entorno huésped respectivamente.

MB_CUR_MAX

Una expresión entera positiva cuyo valor es el número máximo de bytes en un carácter de multibyte para el conjunto de caracteres extendido definido por el escenario actual (categoría `LC_CTYPE`), y cuyo valor nunca es mayor de `MB_LEN_MAX`.

NULL

Una constante de apuntador nulo definida por la puesta en práctica.

RAND_MAX

Una expresión constante de intervalo, cuyo valor es el valor máximo devuelto por la función `rand`. El valor de la macro `RAND_MAX` deberá ser por lo menos 32767.

div_t

Un tipo de estructura que es el tipo de valor devuelto por la función `div`.

ldiv_t

Un tipo de estructura que es el tipo de valor devuelto por la función `ldiv`.

size_t

El tipo entero no signado del resultado del operador **sizeof**.

wchar_t

Un tipo entero cuyo rango de valores puede representar códigos precisos para todos los miembros del conjunto de caracteres extendido más específico entre escenarios soportados; el carácter nulo deberá tener el valor de código cero y cada miembro del conjunto de caracteres básico deberá tener un valor de código igual a su valor cuando se utilice como un carácter solo en una constante de caracteres entera.

double atof(const char *nptr);

Convierte la porción inicial de la cadena a la cual señala **nptr** a representación **double**. La función **atof** devuelve el valor convertido.

int atoi(const char *nptr);

Convierte la porción inicial de la cadena a la cual señala **nptr** a representación **int**. La función **atoi** devuelve el valor convertido.

long int atol(const char *nptr);

Convierte la porción inicial de la cadena a la cual señala **nptr** a representación **long**. La función **atol** devuelve el valor convertido.

double strtod(const char *nptr, char **endptr);

Convierte la porción inicial de la cadena a la cual señala **nptr** a representación **double**. Primero, divide en tres partes la cadena de entrada: una secuencia inicial, posiblemente vacía, de caracteres en blanco (tal y como se define en la función **isspace**), una secuencia sujeto, parecida a una constante de punto flotante; y una cadena final de uno o más caracteres no reconocidos, que incluyen el carácter nulo de terminación de la cadena de entrada. A continuación, intenta convertir la secuencia sujeto a un número de punto flotante y devuelve el resultado.

La forma expandida de la secuencia sujeto es un signo más o un signo menos opcional, a continuación una secuencia de dígitos no vacía, que de forma opcional contenga un carácter de punto decimal, a continuación una parte exponencial opcional, pero sin sufijo flotante. La secuencia sujeto se define como la subsecuencia inicial más extensa de la cadena de entrada, empezando con el primer carácter de espacio no en blanco, esto es de la forma esperada. La secuencia sujeto no contiene ningún carácter, si la cadena de entrada está vacía o está solo formada por espacios en blanco o si el carácter de no espacio en blanco, que está en primer término, es distinto de un signo, de un dígito, o de un carácter de punto decimal.

Si la secuencia sujeto tiene la forma esperada, la secuencia de caracteres que se inicia con el primer dígito o con el carácter de punto decimal (lo que ocurra primero) se interpreta como una constante flotante, excepto que el carácter de punto decimal se utilizará en lugar de un periodo, y que si no aparecen ni una parte exponencial ni un carácter de punto decimal, se supondrá un punto decimal que deberá seguir al último dígito en la cadena. Si la secuencia sujeto empieza con un signo menos, el valor que resulte de la conversión se hace negativa. Un apuntador a la cadena final se almacena en el objeto al cual señala **endptr**, siempre y cuando **endptr** no sea un apuntador nulo.

Si la secuencia sujeto está vacía o no tiene la forma esperada, no se llevará a cabo la conversión; el valor **nptr** se almacenará en el objeto al cual señala **endptr**, siempre y cuando que **endptr** no sea un apuntador nulo.

La función **strtod** devuelve el valor convertido si es que hubiera alguno. Si no se ha ejecutado conversión, devuelve cero. Si el valor correcto queda fuera del rango de los valores representables, se devuelve más o menos **HUGE_VAL** (según el signo del valor), y se almacena el valor de la macro **ERANGE** en **errno**. Si el valor correcto pudiera generar desbordamiento, se devuelve cero y el valor de la macro **ERANGE** queda almacenado en **errno**.

long int strtol(const char *nptr, char **endptr, int base);

Convierte la porción inicial de la cadena a la cual señala **nptr** a **long int**. Primero, descompone la cadena de entrada en tres partes: una secuencia inicial, posiblemente vacía, de caracteres de espacio en blanco (según queda especificado por la función **isspace**), una secuencia sujeto, que se parece a un entero representado en alguna raíz o base determinada por el valor de **base**, y una cadena final, de uno o más caracteres reconocibles, incluyendo el carácter nulo de terminación de la cadena de entrada. A continuación, intenta convertir la secuencia sujeto a un entero, y devuelve el resultado.

Si el valor de **base** es cero, la forma esperada de la secuencia sujeto es la de una constante entera, precedida en forma opcional de un signo más o menos, pero no incluyendo un sufijo entero. Si el valor de **base** es entre 2 y 36, la forma esperada de la secuencia sujeto es una secuencia de letras y dígitos que representan un entero con la raíz o base especificada por **base**, precedida en forma opcional por un signo más o menos, pero sin incluir un sufijo entero. Las letras desde **a** (o **A**) hasta **z** (o **Z**) reciben los valores 10 a 35; sólo se permiten aquellas letras cuyos valores adscritos sean menores que el de la **base**. Si el valor de la **base** es 16, los caracteres **0x** ó **0X** pudieran opcionalmente preceder a la secuencia de letras y de dígitos, a continuación del signo, si es que está presente.

La secuencia sujeto se define como la subsecuencia inicial más larga de la cadena de entrada, empezando a partir del primer carácter que no sea de espacio en blanco, esto es, de la forma esperada.

La secuencia sujeto no contiene ningún carácter, si la cadena de entrada está vacía o está formada sólo de espacios en blanco, o si el primer carácter distinto a espacio en blanco es diferente a un signo o a una letra o dígito permisible.

Si la secuencia sujeto tiene la forma esperada y el valor **base** es cero, la secuencia de caracteres que empieza a partir del primer dígito se interpreta como una constante entera. Si la secuencia sujeto tiene la forma esperada y el valor de la **base** es entre 2 y 36, se utiliza como base para la conversión, dándole a cada letra su valor como se especifica arriba. Si la secuencia sujeto empieza con un signo menos, se colocará un signo menos al valor resultante de la conversión. Un apuntador a la cadena final queda almacenado en el objeto al cual señala **endptr**, siempre y cuando **endptr** no sea nulo.

Si la secuencia sujeto está vacía o no tiene la forma esperada, la conversión no se llevará a cabo; el valor **nptr** se almacenará en el objeto al cual señala **endptr**, siempre y cuando **endptr** no sea un apuntador nulo.

La función **strtol** devuelve el valor convertido, si es que hubiera alguno. Si no se pudo ejecutar ninguna conversión, se devuelve cero. Si el valor correcto queda fuera del rango de valores representables, se devuelve **LONG_MAX** o **LONG_MIN** (de acuerdo al signo del valor), y el valor de la macro **ERANGE** queda almacenado en **errno**.

unsigned long int strtoul(const char *nptr, char **endptr, int base);

Convierte la porción inicial de la cadena a la cual señala **nptr** a una representación **unsigned long int**. La función **strtoul** funciona de forma idéntica a la función **strtol**. La función **strtoul** devuelve el valor convertido, si es que hay alguno. Se devuelve cero, si no se pudo llevar a cabo la conversión. Si el valor correcto queda por fuera del rango de los valores representables, se devuelve **ULONG_MAX**, y se almacena el valor de la macro **ERANGE** en **errno**.

int rand(void);

La función **rand** calcula una secuencia de enteros pseudo aleatorios de rango 0 hasta **RAND_MAX**. La función **rand** devuelve un entero pseudo aleatorio.

void srand(unsigned int seed);

Utiliza el argumento como semilla para una nueva secuencia de números pseudo aleatorios, a devolverse en subsecuentes llamadas a **rand**. Si entonces es llamada a **srand** con el mismo valor de semilla, la secuencia de los números pseudo aleatorios se repetirá. Si se llama a **rand** antes de cualquier llamada a **srand**, la misma secuencia será generada como si se hubiera llamado primero a **srand** con un valor de semilla de 1. La siguientes funciones definen una puesta en práctica portátil de **rand** y de **srand**.

```
static unsigned long int next = 1;
```

```
int rand(void) /* RAND_MAX assumed to be 32767 */
{
    next = next * 1103515245 + 12345;
    return (unsigned int) (next/65536) % 32768;
}

void srand(unsigned int seed)
{
    next = seed;
}
```

void *calloc(size_t nmemb, size_t size);

Asigna espacio para un arreglo de objetos **nmemb**, cada uno de los cuales tiene un tamaño de **size**. El espacio es inicializado para todos los bits cero. La función **calloc** devuelve al espacio asignado un apuntador nulo o un apuntador.

void free(void *ptr);

Hace que sea desasignado el espacio al cual señala **ptr**, esto es, se deje disponible para otra asignación. Si **ptr** es un apuntador nulo, no ocurrirá ninguna acción. De lo contrario, si el argumento no coincide con un apuntador ya devuelto por las funciones **calloc**, **malloc** o **realloc**, o si el espacio ya ha sido desasignado mediante una llamada a **free** o **realloc**, el comportamiento queda indefinido.

void *malloc(size_t size);

Asigna espacio para un objeto cuyo tamaño queda especificado por **size** y cuyo valor es indeterminado. La función **malloc** devuelve un apuntador nulo o un apuntador al espacio asignado.

void *realloc(void *ptr, size_t size);

Modifica el tamaño del objeto al cual señala **ptr** al tamaño definido por **size**. El contenido del objeto deberá conservarse sin modificar hasta el menor de entre los tamaños nuevo y viejo. Si el nuevo tamaño es mayor, el valor de la porción recién asignada del objeto queda indeterminada. Si **ptr** es un apuntador nulo, la función **realloc** se comporta como la función **malloc** para el tamaño especificado. De lo contrario, si **ptr** no encuentra un apuntador devuelto por las funciones **calloc**, **malloc**, o **realloc**, o si el espacio ha sido ya desasignado mediante una llamada a las funciones **free** o **realloc**, el comportamiento queda indefinido. Si el espacio no puede ser asignado, el objeto al cual apunta **ptr** queda sin modificación. Si **size** es cero y **ptr** no es nulo, el objeto al cual señala queda liberado. La función **realloc** devuelve o un apuntador nulo o un apuntador al espacio asignado posiblemente relocado.

void abort(void);

Hace que ocurra una terminación anormal de programa, a menos de que se detecte una señal **SIGABRT** y el manejador de señal no se devuelva. Queda dependiente de la puesta en marcha si se vaciarán los flujos de salida abiertos o si se cerrarán los flujos abiertos y serán eliminados los archivos temporales. Mediante la llamada de función **raise** (**SIGABRT** se devuelve al entorno huésped una forma dependiente de la puesta en práctica del estado de *terminación no exitoso*). La función **abort** no puede devolver a su llamador.

int atexit(void (*func)(void));

Registra la función a la cual apunta **func**, para que a la terminación normal de un programa ésta sea llamada sin argumentos. La puesta en práctica debe aceptar el registro de por lo menos 32 funciones. La función **atexit** devuelve cero si el registro tiene éxito, y no cero si falla.

void exit(int status);

Hace que ocurra una terminación normal del programa. Si el programa ejecuta más de una llamada a la función **exit**, el comportamiento queda indefinido. Primero, serán llamadas todas las funciones registradas por la función **atexit**, en orden inverso a la de su registro. Cada función será llamada

tantas veces como fue registrada. A continuación, serán vaciados todos los flujos abiertos con datos en búfer no escritos, serán cerrados todos los flujos abiertos, y serán eliminados todos los archivos creados mediante la función **tempfile**.

Por último, el control será devuelto al entorno huésped. Si el valor de **status** es cero o **EXIT_SUCCESS**, será devuelta una forma, definida por la puesta en práctica, del estado de *terminación con éxito*. Si el valor de **status** es **EXIT_FAILURE**, será devuelta una forma, definida por la puesta en práctica, del estado *terminación sin éxito*. De lo contrario, el estado devuelto queda definido por la puesta en práctica. La función **exit** no puede devolver a su llamador.

char *getenv(const char *name);

Busca en una *lista de entorno*, proporcionada por el entorno huésped, una cadena que coincida con la cadena a la cual apunta **name**. El conjunto de nombres de entorno y el método para modificar la lista de entorno, quedan definidas por la puesta en práctica. Devuelve un apuntador a una cadena asociada con el miembro de lista coincidente. La cadena a la cual apunta no será modificada por el programa, pero puede ser sobrescrita por una llamada subsiguiente a la función **getenv**. Si no puede ser encontrado el **name** especificado, devuelve un apuntador nulo.

int system(const char *string);

Pasa la cadena a la cual apunta **string** al entorno huésped para que sea ejecutada por un *procesador de comandos* de una forma definida por la puesta en práctica. Se puede utilizar un apuntador nulo en lugar de **string** para indagar si existe el procesador de comandos. Si el argumento es un apuntador nulo, la función **system** devolverá no cero sólo en el caso de que el procesador de comandos esté disponible. Si el argumento no es un apuntador nulo, la función **system** devolverá un valor definido por la puesta en práctica.

void *bsearch(const void *key, const void *base, size_t nmemb, size_t size, int (*compar)(const void *, const void *));

Busca en un arreglo de objetos **nmemb**, el elemento inicial al cual señala **base**, en busca de un elemento que coincida con el objeto al cual señala **key**. El tamaño de cada elemento del arreglo se especifica mediante **size**. Se llama a la función de comparación a la cual señala **compar**, con dos argumentos que apuntan al objeto **key** y a un elemento del arreglo, en este orden. La función devolverá un entero menor que, igual que o mayor que cero si el objeto **key** se considera, respectivamente, ser menor que, coincidir o ser mayor que el elemento del arreglo. El arreglo debe consistir de: todos los elementos que se comparan como menores que, todos los elementos que se comparan como iguales y todos los elementos que se comparan como mayores que el objeto **key**, en ese orden.

La función **bsearch** devuelve un apuntador a un elemento coincidente del arreglo, o un apuntador nulo, si no encuentra coincidencia. Si dos elementos son comparados como iguales, el elemento coincidente queda indeterminado.

void qsort(void *base, size_t nmemb, size_t size, int (*compar)(const void *, const void *));

Ordena un arreglo de **nmemb** objetos. El elemento inicial es el que señala a **base**. El tamaño de cada objeto está especificado por **size**. El contenido del arreglo es clasificado u ordenado en orden ascendente, de acuerdo con la función de comparación a la cual señala **compar**, misma que es llamada con dos argumentos, que apuntan a los objetos bajo comparación. La función devuelve un entero menor que o igual, o mayor que cero si el primer argumento se considera ser respectivamente menor que, igual a o mayor que el segundo. Si los dos elementos que se comparan son iguales, su orden en el arreglo clasificado queda indefinido.

int abs(int j);

Calcula el valor absoluto de un entero **j**. Si el resultado no puede ser representado, el comportamiento queda indefinido. La función **abs** devuelve el valor absoluto.

div_t div(int numer, int denom);

Calcula el cociente y residuo o módulo de la división del numerador `numer` entre el denominador `denom`. Si la división es inexacta, el cociente resultante es el entero de menor magnitud que resulte más cercano al cociente algebraico. Si el resultado no puede ser representado, el comportamiento queda indefinido; de lo contrario, `quot * denom + rem` debe ser igual a `numer`. La función `div` devuelve una estructura del tipo `div_t`, que comprende tanto el cociente como el residuo. La estructura debe contener los miembros siguientes, en cualquier orden:

```
int quot;    /* quotient */
int rem;     /* remainder */
```

```
long int labs(long int j);
```

Similar a la función `abs`, excepto que el argumento y el valor devuelto cada uno de ellos tiene el tipo `long int`.

```
ldiv_t ldiv(long int numer, long int denom);
```

Similar a la función `div`, excepto que el argumento y los miembros de la estructura devuelta (que tiene el tipo `ldiv_t`) todos ellos tienen el tipo `long int`.

```
int mblen(const char *s, size_t n);
```

Si `s` no es un apuntador nulo, la función `mblen` determina el número de bytes contenidos en el carácter multibyte al cual señala `s`. Si `s` es un apuntador nulo, la función `mblen` devuelve un valor no cero o cero, si las codificaciones de caracteres de multibyte, respectivamente tienen o no, codificaciones dependientes del estado. Si `s` no es un apuntador nulo, la función `mblen` devuelve cero (si `s` apunta al carácter nulo) o devuelve el número de bytes contenida en el carácter de multibyte (si los siguientes `n` o menos bytes forman un carácter de multibyte válidos), o devuelve -1 (si no forman un carácter de multibyte válido).

```
int mbtowc(wchar_t *pwc, const char *s, size_t n);
```

Si `s` no es un apuntador nulo, la función `mbtowc` determina el número de bytes contenidos en el carácter de multibyte al cual señala `s`. A continuación determina el código para el valor del tipo `wchar_t` que corresponda a dicho carácter de multibyte. (El valor del código que corresponda al carácter nulo es cero). Si el carácter de multibyte es válido y `pwc` no es un apuntador nulo, la función `mbtowc` almacena el código en el objeto al cual apunta `pwc`. Por lo menos `n` bytes del arreglo al cual apunta `s` serán examinados.

Si `s` es un apuntador nulo, la función `mbtowc` devuelve un valor no cero o cero, si las codificaciones de caracteres de multibyte, en forma respectiva tienen o no tienen codificaciones que dependan del estado. Si `s` no es un apuntador nulo, la función `mbtowc` devuelve cero (si `s` señala al carácter nulo) o devuelve el número de bytes que están contenidas en el carácter de multibyte convertido (si la `n` siguiente o menos bytes forman un carácter de varios bytes válidos), o devuelve -1 (si no forman un carácter de multibyte válido). En ninguno de los casos el valor devuelto será mayor que `n` o el valor de la macro `MB_CUR_MAX`.

```
int wctomb(char *s, wchar_t wchar);
```

La función `wctomb` determina el número de bytes necesarios para representar un carácter de multibyte correspondiendo al código cuyo valor es `wchar` (incluyendo cualquier modificación en estado de desplazamiento). Almacena la representación de caracteres de multibyte en el objeto de arreglo al cual apunta `s` (si `s` no es un apuntador nulo). Como máximo se almacenan `MB_CUR_MAX` caracteres. Si el valor de `wchar` es cero, la función `wctomb` se deja en el estado de desplazamiento inicial.

Si `s` es un apuntador nulo, la función `wctomb` devuelve un valor no cero o cero, en el caso que las codificaciones de caracteres de multibyte, respectivamente tengan o no tengan codificaciones dependiendo del estado. Si `s` no es un apuntador nulo, la función `wctomb` devuelve -1 si el valor de `wchar` no corresponde a un carácter de multibyte válido o devuelve el número de bytes contenidos en el carácter de multibyte correspondiente al valor de `wchar`. En ningún caso el valor devuelto será mayor que el valor de la macro `MB_CUR_MAX`.

```
size_t mbstowcs(wchar_t *pwcs, const char *s, size_t n);
```

La función `mbstowcs` convierte una secuencia de caracteres multibyte que empieza en el estado de desplazamiento inicial del arreglo al cual apunta `s` en una secuencia de códigos correspondientes y almacena no más de `n` códigos en el arreglo al cual señala `pwcs`. No se examinarán o convertirán caracteres de multibyte que sigan después de un carácter nulo (mismo que se convierte a un código con valor cero). Cada carácter multibyte se convierte igual que en el caso de una función `mbtowc`, excepto de que no se afecta el estado de desplazamiento de la función `mbtowc`.

No se modificarán más de `n` elementos en el arreglo al cual señala `pwcs`. El comportamiento de copiar entre objetos que se superpongan queda indefinido. Si se encuentra un carácter de multibyte inválido, la función `mbstowcs` devuelve `(size_t) - 1`. De no ser así, la función `mbstowcs` devuelve el número de elementos de arreglos modificados, sin incluir el código cero de terminación, si es que hay alguno.

```
size_t wcstombs(char *s, const wchar_t *pwcs, size_t n);
```

La función `wcstombs` convierte una secuencia de códigos que corresponden a caracteres de multibyte del arreglo al cual apunta `pwcs` en una secuencia de caracteres de multibyte que empieza en el estado inicial de desplazamiento y almacena estos caracteres de multibyte en el arreglo al cual señala `s`, deteniéndose si un carácter multibyte se excediese del límite total de bytes igual a `n` o si se almacena un carácter nulo. Cada código se convierte igual que en el caso de una llamada a una función `wctomb`, a excepción de que el estado de desplazamiento de la función `wctomb` no se afecta.

No se modificarán más de `n` bytes en el arreglo al cual señala `s`. Si la copia ocurre entre objetos que se superponen, el comportamiento queda indefinido. Si se encuentra con un código que no corresponda con un carácter válido de multibyte, la función `wcstombs` devuelve `(size_t) - 1`. De lo contrario, la función `wcstombs` devuelve el número de bytes modificados, sin incluir el carácter nulo de terminación, si existiese.

B.12 Manejo de cadenas <string.h>

NULL

Una constante de apuntador nula, definida por la puesta en práctica.

size_t

El tipo integral no signado resultante del operador `sizeof`.

```
void *memcpy(void *s1, const void *s2, size_t n);
```

La función `memcpy` copia `n` caracteres del objeto al cual señala `s2` al objeto al cual señala `s1`. Si la copia tiene lugar entre objetos que se superponen, el comportamiento queda indefinido. La función `memcpy` devuelve el valor de `s1`.

```
void *memmove(void *s1, const void *s2, size_t n);
```

La función `memmove` copia `n` caracteres del objeto al cual señala `s2` en el objeto al cual señala `s1`. La copia se lleva a cabo como si los `n` caracteres del objeto al cual señala `s2` se copian primero a un arreglo temporal de `n` caracteres, que no se superponen sobre los objetos al cual señala `s1` y `s2`, y a continuación los `n` caracteres del arreglo temporal se copian al objeto al que señala `s1`. La función `memmove` devuelve el valor de `s1`.

```
char *strcpy(char *s1, const char *s2);
```

La función `strcpy` copia la cadena a la cual señala `s2` (incluyendo el carácter nulo de terminación) al arreglo al cual señala `s1`. Si la copia se lleva a cabo entre objetos que se superponen, el comportamiento queda indefinido. La función `strcpy` devuelve el valor de `s1`.

```
char *strncpy(char *s1, const char *s2, size_t n);
```

La función `strncpy` copia no más de `n` caracteres, (caracteres que sigan a un carácter nulo ya no son copiados) del arreglo al cual señala `s2` al arreglo al cual señala `s1`. Si la copia se lleva a cabo entre objetos que se superponen, el comportamiento queda indefinido. Si el arreglo al cual señala `s2` es una cadena más corta de `n` caracteres, se agregarán caracteres nulos a la copia en el arreglo al cual señala `s1`, hasta que en total se hayan escrito `n` caracteres. La función `strncpy` devuelve el valor de `s1`.

`char *strcat(char *s1, const char *s2);`

La función `strcat` agrega una copia de la cadena a la cual señala `s2` (incluye el carácter nulo de terminación) al final de la cadena a la cual señala `s1`. El carácter inicial de `s2` sobrescribe el carácter nulo existente al final de `s1`. Si la copia se lleva a cabo entre objetos que se superponen, el comportamiento queda indefinido. La función `strcat` devuelve el valor de `s1`.

`char *strncat(char *s1, const char *s2, size_t n);`

La función `strncat` agrega no más de `n` caracteres (el carácter nulo y los caracteres que le sigan no serán agregados) del arreglo al cual apunta `s2` al final de la cadena a la cual apunta `s1`. El carácter inicial de `s2` sobrescribe el carácter nulo al final de `s1`. Siempre se agregará un carácter nulo de terminación al resultado. Si la copia se lleva a cabo entre objetos que se superponen, el comportamiento queda indefinido. La función `strncat` devuelve el valor de `s1`.

`int memcmp(const void *s1, const void *s2, size_t n);`

La función `memcmp` compara los primeros `n` caracteres del objeto al cual señala `s1` con los primeros `n` caracteres del objeto al cual señala `s2`. La función `memcmp` devuelve un entero mayor que, igual a o menor que cero, según si el objeto al cual señala `s1` es mayor que, igual a o menor que el objeto al cual señala `s2`.

`int strcmp(const char *s1, const char *s2);`

La función `strcmp` compara la cadena a la cual señala `s1` con la cadena a la cual señala `s2`. La función `strcmp` devuelve un entero mayor que, igual a o menor que cero, según que la cadena a la cual señala `s1` sea mayor que, igual a o menor que la cadena a la cual señala `s2`.

`int strcoll(const char *s1, const char *s2);`

La función `strcoll` compara la cadena a la cual señala `s1` con la cadena a la cual señala `s2`, ambas interpretadas como apropiadas a la categoría `LC_COLLATE` del escenario actual. La función `strcoll` devuelve un entero mayor que, igual a o menor que cero, según que la cadena a la cual señala `s1` sea mayor que, igual a o menor que la cadena a la cual señala `s2` cuando ambos están interpretadas como apropiadas para el escenario actual.

`int strncmp(const char *s1, const char *s2, size_t n);`

La función `strncmp` compara no más de `n` caracteres (los caracteres que sigan a un carácter nulo no son comparados) del arreglo al cual señala `s1` con el arreglo al cual señala `s2`. La función `strncmp` devuelve un entero mayor que, igual a o menor que cero, según si el arreglo, posiblemente terminado en nulo, al cual señala `s1`, es mayor que, igual a o menor que el arreglo, posiblemente terminado en nulo, al cual señala `s2`.

`size_t strxfrm(char *s1, const char *s2, size_t n);`

La función `strxfrm` transforma la cadena a la cual señala `s2` y coloca la cadena resultante en el arreglo al cual señala `s1`. La transformación es tal que si la función `strcmp` se aplica a las dos cadenas transformadas, devuelve un valor mayor que igual a o menor que cero, en concordancia con el resultado de la función `strcoll` aplicable a las dos cadenas originales. No se colocan más de `n` caracteres en el arreglo resultante al cual señala `s1`, incluyendo el carácter nulo de terminación. Si `n` es cero, `s1` es posible que sea un apuntador nulo. Si la copia se lleva acabo entre objetos que se superponen, el comportamiento queda indefinido. La función `strxfrm` devuelve la longitud de la cadena transformada (sin incluir el carácter nulo de terminación). Si el valor es `n` o más, el contenido del arreglo al cual señala `s1` queda indeterminado.

`void *memchr(const void *s, int c, size_t n);`

La función `memchr` localiza la primera instancia de `c` (convertida a un `unsigned char`) en los `n` caracteres iniciales (cada uno de ellos interpretados como un `unsigned char`) del objeto al cual señala `s`. La función `memchr` devuelve un apuntador al carácter localizado, o un apuntador nulo si el carácter no existe en el objeto.

`char *strchr(const char *s, int c);`

La función `strchr` localiza la primera instancia de `c` (convertido a un `char`) en la cadena a la cual señala `s`. El carácter nulo de terminación se considera como parte de la cadena. La función `strchr` devuelve un apuntador al carácter localizado, o un apuntador nulo si el carácter no está incluido en la cadena.

`size_t strcspn(const char *s1, const char *s2);`

La función `strcspn` calcula la longitud del segmento inicial máximo de la cadena a la cual señala `s1` que está formada por completo de caracteres que no sean de la cadena a la cual señala `s2`. La función `strcspn` devuelve la longitud del segmento.

`char *strpbrk(const char *s1, const char *s2);`

La función `strpbrk` localiza la primera instancia de la cadena a la cual señala `s1` de cualquier carácter de la cadena a la cual señala `s2`. La función `strpbrk` devuelve un apuntador al carácter, o un apuntador nulo si ningún carácter de `s2` ocurre en `s1`.

`char *strrchr(const char *s, int c);`

La función `strrchr` localiza la última instancia de `c` (convertido a `char`) en la cadena a la cual señala `s`. El carácter nulo de terminación se considera como parte de la cadena. La función `strrchr` devuelve un apuntador al carácter, o un apuntador nulo si `c` no está incluido en la cadena.

`size_t strspn(const char *s1, const char *s2);`

La función `strspn` calcula la longitud del segmento inicial máximo de la cadena a la cual señala `s1`, que consista en totalidad de caracteres de la cadena a la cual señala `s2`. La función `strspn` devuelve la longitud del segmento.

`char *strstr(const char *s1, const char *s2);`

La función `strstr` localiza la primera instancia en la cadena a la cual señala `s1` de la secuencia de caracteres (excluyendo el carácter nulo de terminación) en la cadena a la cual señala `s2`. La función `strstr` devuelve un apuntador a la cadena localizada, o un apuntador nulo si la cadena no se encuentra. Si `s2` señala a una cadena de longitud cero, la función devuelve `s1`.

`char *strtok(char *s1, const char *s2);`

Una secuencia de llamadas a función `strtok` divide la cadena a la cual señala `s1` en una secuencia de componentes léxicos, cada uno de los cuales queda delimitado por un carácter de la cadena a la cual señala `s2`. La primera llamada en la secuencia tiene como argumento `s1`, y es seguida por llamadas con un apuntador nulo como primer argumento. La cadena separadora a la cual señala `s2` puede cambiar de llamada a llamada.

La primera llamada en la secuencia busca en la cadena a la cual señala `s1` el primer carácter que no esté contenido en la cadena separadora actual a la cual señala `s2`. Si dicho carácter no se encuentra, entonces no existen componentes léxicos en la cadena a la cual señala `s1` y la función `strtok` devuelve un apuntador nulo. Si se encuentra dicho carácter, ese es el inicio del primer componente léxico.

La función `strtok` a continuación busca a partir de ahí un carácter que esté contenido en la cadena separadora actual. Si no encuentra dicho carácter, el componente léxico actual se extiende hasta el fin de la cadena a la cual señala `s1`, y las búsquedas subsecuentes para un componente léxico devolverán un apuntador nulo. Si se encuentra dicho carácter, queda sobrescrito por un carácter nulo, lo cual da por terminado el componente léxico actual. La función `strtok` guarda un apuntador al carácter siguiente, a partir del cual se iniciará el siguiente componente léxico.

Cada llamada subsiguiente, con un apuntador nulo como valor de su primer argumento, empieza a buscar a partir del apuntador guardado y se comporta como se describió anteriormente. La puesta en práctica debe comportarse como si ninguna función de biblioteca llama a la función `strtok`. La función `strtok` devuelve un apuntador al primer carácter de un componente léxico, o a un apuntador nulo si tal componente no existe.

`void *memset(void *s, int c, size_t n);`
La función `memset` copia el valor de `c` (convertido a un `unsigned char`) a cada uno de los primeros `n` caracteres en el objeto al cual señala `s`. La función `memset` devuelve el valor de `s`.

`char *strerror(int errnum);`
La función `strerror` relocaliza el número de error en `errnum` a una cadena de mensaje de error. La puesta en práctica deberá comportarse como si ninguna función de biblioteca llama a la función `strerror`. La función `strerror` devuelve un apuntador a la cadena, el contenido de la cual está definido por la puesta en práctica. El arreglo al cual se apunta no debe modificarse por el programa, que pudiera ser sobrescrito debido a una llamada subsiguiente a la función `strerror`.

`size_t strlen(const char *s);`
La función `strlen` computa la longitud de la cadena a la cual señala `s`. La función `strlen` devuelve el número de caracteres que anteceden al carácter nulo de terminación.

B.13 Fecha y hora <time.h>

`CLOCKS_PER_SEC`
El número por segundo del valor devuelto por la función `clock`.

`NULL`
Constante de apuntador nula, definida por la puesta en práctica.

`clock_t`
Un tipo aritmético capaz de representarse la fecha.

`time_t`
Un tipo aritmético capaz de representarse la hora.

`size_t`
Un tipo entero no signado del resultado del operador `sizeof`.

`struct tm`
Contiene los componentes de una hora de calendario, conocida como *hora desglosada*. La estructura deberá contener por lo menos los siguientes miembros, en cualquier orden. La semántica de los miembros y sus rangos normales se expresan en los comentarios.

```
int tm_sec;      /* seconds after the minute—[0,61] */
int tm_min;      /* minutes after the hour—[0,59] */
int tm_hour;     /* hours since midnight—[0,23] */
int tm_mday;     /* day of the month—[1,31] */
int tm_mon;      /* months since January—[0,11] */
int tm_year;     /* year since 1900 */
int tm_wday;     /* days since Sunday—[0,6] */
int tm_yday;     /* days since January 1—[0,365] */
int tm_isdst;    /* Daylight Saving Time flag */
```

El valor `tm_isdst` es positivo si está en efecto Daylight Saving Time, es cero si no está en efecto Daylight Saving Time, y negativo si la información no está disponible.

`clock_t clock(void)`
La función `clock` determina la hora de procesador utilizada. La función `clock` devuelve la mejor aproximación de la puesta en práctica de la hora del procesador utilizada por el programa desde el principio de una era, definida por la puesta en práctica, únicamente relacionada con la invocación del programa. Para determinar la hora en segundos, el valor devuelto por la función `clock` deberá ser dividido por el valor de la macro `CLOCKS_PER_SEC`. Si la hora del procesador utilizada no está disponible o su valor no puede ser representado, la función devuelve el valor `(clock_t) - 1`.

`double difftime(time_t time1, time_t time0);`
La función `difftime` calcula la diferencia entre dos horas de calendario: `time1 - time0`. La función `difftime` devuelve la diferencia, expresada en segundos, como un `double`.

`time_t mktime(struct tm *timeptr);`
La función `mktime` convierte la hora desglosada, expresada como hora local, en la estructura a la cual señala `timeptr` en un valor de hora de calendario, con la misma codificación que la de los valores devueltos por la función `time`. Serán ignorados los valores originales de los componentes `tm_wday` y `tm_yday` de la estructura, y los valores originales de los demás componentes no quedarán restringidos a los rangos arriba indicados. A la terminación exitosa, se ajustarán apropiadamente los valores de los componentes `tm_wday` y `tm_yday` y los demás componentes se ajustarán para representar la hora de calendario específico, pero con sus valores forzados a los rangos arriba indicados; el valor final de `tm_mday` no se define hasta que estén determinados `tm_mon` y `tm_year`. La función `mktime` devuelve la hora especificada de calendario codificada como un valor del tipo `time_t`. Si la hora de calendario no puede representarse, la función devuelve el valor `(time_t) - 1`.

`time_t time(time_t *timer);`
La función `time` determina la hora de calendario actual. La función `time` devuelve la mejor aproximación según la puesta en práctica de la hora de calendario actual. El valor `(time_t) - 1` será devuelto si dicha hora de calendario no está disponible. Si `timer` no es un apuntador nulo, el valor devuelto también se asignará al objeto al cual apunta.

`char *asctime(const struct tm *timeptr);`
La función `asctime` convierte la hora desglosada en la estructura a la cual señala `timeptr` en una cadena de la forma
`Sun Sep 16 01:03:52 1973\n\n`
La función `asctime` devuelve un apuntador a la cadena.

`char *ctime(const time_t *timer);`
La función `ctime` convierte la hora de calendario al cual señala `timer` a hora local en forma de una cadena. Es equivalente a
`asctime(localtime(timer))`
La función `ctime` devuelve el apuntador devuelto por la función `asctime` con dicha hora desglosada como argumento.

`struct tm *gmtime(const time_t *timer);`
La función `gmtime` convierte la hora de calendario al cual señala `timer` en una hora desglosada, expresada en forma de Coordinated Universal Time (UTC) La función `gmtime` devuelve un apuntador a dicho objeto, o un apuntador nulo si UTC no está disponible.

`struct tm *localtime(const time_t *timer);`
La función `localtime` convierte la hora de calendario al cual señala `timer` en una hora desglosada, expresada en forma de hora local. La función `localtime` devuelve un apuntador a dicho objeto.

`size_t strftime(char *s, size_t maxsize, const char *format, const struct tm *timeptr);`

La función `strftime` coloca caracteres en el arreglo al cual señala `s` controlados por la cadena a la cual señala `format`. La cadena `format` consiste de un cero o más especificadores de conversión y caracteres ordinarios de varios bytes. Todos los caracteres ordinarios (incluyendo el carácter nulo de terminación) serán copiados al arreglo sin modificarse. Si la copia se hace entre objetos que se superponen, el comportamiento quedará indefinido. No se colocarán más caracteres que los especificados en `maxsize` en el arreglo. Cada especificador de conversión será remplazado por los caracteres apropiados tal y como se describe en la lista siguiente. Los caracteres apropiados quedan determinados por la categoría `LC_TIME` del escenario actual y por los valores contenidos en la estructura a la cual señala `timeptr`.

- `%a` es remplazado por la abreviatura del nombre del día de la semana del escenario.
- `%A` es remplazado por el nombre completo del día de la semana del escenario.
- `%b` es remplazado por la abreviatura del nombre del mes del escenario.
- `%B` es remplazado por el nombre completo del mes del escenario.
- `%c` es remplazado por la representación apropiada de fecha y de hora del escenario.
- `%d` es remplazado por el día del mes como número decimal (01 - 31).
- `%H` es remplazado por la hora (reloj de 24 horas) como número decimal (00 - 23).
- `%I` es remplazado por la hora (reloj de 12 horas) como número decimal (01 - 12).
- `%j` es remplazado por el día del año como número decimal (001 - 366).
- `%m` es remplazado por el mes como número decimal (01 - 12).
- `%M` es remplazado por el minuto como número decimal (00 - 59).
- `%p` es remplazado por el equivalente para el escenario de las designaciones AM/PM asociadas con un reloj de 12 horas.
- `%S` es remplazado como segundos como número decimal (00 - 61).
- `%U` es remplazado por el número de la semana correspondiente del año (siendo el primer domingo el primer día de la semana 1) como número decimal (00 - 53).
- `%w` es remplazado por el día de la semana como número decimal (0 - 6), donde domingo es 0.
- `%W` es remplazado por el número de la semana correspondiente del año (el primer lunes es el primer día de la semana 1) como número decimal (00 - 53).
- `%x` es remplazado por la representación apropiada de la fecha, de acuerdo con el escenario.
- `%X` es remplazado por la representación apropiada de la hora para el escenario.
- `%y` es remplazado por el año, sin siglos, como un número decimal (00 - 99).
- `%Y` es remplazado por el año, incluyendo siglos, como un número decimal.
- `%Z` es remplazado por el nombre o abreviatura de la zona de tiempo, y mediante ningún carácter, si no hay determinable ninguna zona de tiempo.
- `%%` es remplazado por `%`.

Si el especificador de conversión no es ninguno de los arriba citados, el comportamiento queda indefinido. Si el número total de caracteres resultantes, incluyendo el carácter nulo de terminación no es más de `max-size`, la función `strftime` devuelve el número de carácter colocados en el arreglo al cual señala `s`, sin incluir el carácter nulo de terminación. De lo contrario, se devuelve cero y el contenido del arreglo queda indeterminado.

B.14 Límites de puesta en práctica

`<limits.h>`

Los siguientes deben ser definidos en magnitud (valor absoluto) como igual a o mayor que los valores siguientes.

- `#define CHAR_BIT` 8
El número de bits para el objeto más pequeño que no sea un campo de bits (byte).
- `#define SCHAR_MIN` -127
El valor mínimo para un objeto del tipo `signed char`.
- `#define SCHAR_MAX` +127
El valor máximo para un objeto del tipo `signed char`.
- `#define UCHAR_MAX` 255
El valor máximo para un objeto del tipo `unsigned char`.
- `#define CHAR_MIN` 0 ó `SCHAR_MIN`
El valor mínimo para un objeto del tipo `char`.
- `#define CHAR_MAX` `UCHAR_MAX` ó `SCHAR_MAX`
El valor máximo para un objeto del tipo `char`.
- `#define MB_LEN_MAX` 1
El número máximo de bytes de un carácter multibyte, para cualquier escenario soportado.
- `#define SHRT_MIN` -32767
El valor mínimo para un objeto del tipo `short int`.
- `#define SHRT_MAX` +32767
El valor máximo para un objeto del tipo `short int`.
- `#define USHRT_MAX` 65535
El valor máximo para un objeto del tipo `unsigned short int`.
- `#define INT_MIN` -32767
El valor mínimo para un objeto del tipo `int`.
- `#define INT_MAX` +32767
El valor máximo para un objeto del tipo `int`.
- `#define UINT_MAX` 65535
El valor máximo para un objeto del tipo `unsigned int`.
- `#define LONG_MIN` -2147483647
El valor mínimo para un objeto del tipo `long int`.
- `#define LONG_MAX` +2147483647
El valor máximo para un objeto del tipo `long int`.
- `#define ULONG_MAX` 4294967295
El valor máximo para un objeto del tipo `unsigned long int`.

`<float.h>`

- `#define FLT_ROUNDS`
El modo de redondeo para la suma en punto flotante
 - 1 indeterminable
 - 0 hacia cero

- 1

hacia el más cercano
- 2

hacia infinito positivo
- 3

hacia infinito negativo

Lo siguiente debe ser definido en magnitud (valor absoluto) igual a o mayor que a los valores siguientes.

```
#define FLT_RADIX                2
    La raíz de la representación exponencial, b.

#define FLT_MANT_DIG
#define LDBL_MANT_DIG
#define DBL_MANT_DIG
    El número de base FLT_RADIX dígitos en el significando del punto flotante p.

#define FLT_DIG                6
#define DBL_DIG                10
#define _LDB_DIG                10
    El número de dígitos decimales q, tal que cualquier número de punto flotante con q dígitos decimales
    pueda ser redondeado a un número de punto flotante con p raíz b dígitos y de regreso otra vez sin
    modificación a los dígitos decimales q.

#define FLT_MIN_EXP
#define DBL_MIN_EXP
#define LDBL_MIN_EXP
    El entero negativo mínimo tal que FLT_RADIX elevado a dicha potencia menos 1 es un número de
    punto flotante normalizado.

#define FLT_MIN_10_EXP        - 37
#define DBL_MIN_10_EXP        - 37
#define LDBL_MIN_10_EXP        - 37
    El entero negativo mínimo tal que 10 elevado a dicha potencia está en el rango de los números de punto
    flotante normalizados.

#define FLT_MAX_EXP
#define DBL_MAX_EXP
#define LDBL_MAX_EXP
    El entero máximo tal que FLT_RADIX elevado a dicha potencia menos 1 es un número finito de punto
    flotante representable.

#define FLT_MIN_10_EXP        + 37
#define DBL_MIN_10_EXP        + 37
#define LDBL_MIN_10_EXP        + 37
    El entero máximo tal que 10 elevado a esa potencia está en el rango de los números de punto flotante
    finitos y representables.
```

Lo siguiente deberá ser definido igual a o mayor que los valores mostrados a continuación.

```
#define FLT_MAX                1E+37
#define DBL_MAX                1E+37
#define LDBL_MAX                1E+37
    El número de punto flotante finito representable máximo.
```

Lo siguiente debe ser definido igual a o menor que los valores mostrados a continuación.

```
#define FLT_EPSILON            1E-5
```

```
#define DBL_EPSILON            1E-9
#define LDBL_EPSILON            1E-9
    La diferencia entre 1.0 y el valor mínimo mayor que 1.0 que sea representable en el tipo dado de punto
    flotante.

#define FLT_MIN                1E-37
#define DBL_MIN                1E-37
#define LDBL_MIN                1E-37
    El número de punto flotante positivo normalizado mínimo.
```

Apéndice C

Precedencia y asociatividad de operadores

Operador	Asociatividad
() [] -> .	izquierda a derecha
++ -- + - ! ~ (tipo) * & sizeof	derecha a izquierda
* / %	izquierda a derecha
+ -	izquierda a derecha
<< >>	izquierda a derecha
< <= > >=	izquierda a derecha
== !=	izquierda a derecha
&	izquierda a derecha
^	izquierda a derecha
	izquierda a derecha
&&	izquierda a derecha
	izquierda a derecha
?:	derecha a izquierda
= += -= *= /= %= &= ^= = <<= >>=	derecha a izquierda
,	izquierda a derecha

Los operadores aparecen de la parte superior a la inferior en orden decreciente de precedencia.

Apéndice D

Conjunto de caracteres ASCII

	0	1	2	3	4	5	6	7	8	9
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht
1	nl	vt	ff	cr	so	si	dle	dc1	dc2	dc3
2	dc4	nak	syn	etb	can	em	sub	esc	fs	gs
3	rs	us	sp	!	"	#	\$	%	&	'
4	(	)	*	+	,	-	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	_	`	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{		}	~	del		

Los dígitos a la izquierda de la tabla son los dígitos izquierdos del equivalente decimal del código de caracteres, y los dígitos en la parte superior de la tabla son los dígitos derechos del código de caracteres. Por ejemplo, el código de carácter para 'F' es 70, y el correspondiente para & es 38.

APÉNDICE E

Sistemas numéricos

Objetivos

- Comprender los conceptos básicos de los sistemas numéricos como base, valor posicional y valor simbólico.
- Comprender cómo trabajar con números representados en sistemas numéricos binario, octal, y hexadecimal.
- Ser capaz de reducir los números binarios a números octales o a números hexadecimales.
- Ser capaz de convertir los números octales y hexadecimales a números binarios.
- Ser capaz de convertir en ambos sentidos entre números decimales y sus equivalentes binario, octal y hexadecimal.
- Comprender la aritmética binaria y cómo se representan los números binarios negativos mediante la notación de complemento a dos.

He aquí sólo números ratificados.
William Shakespeare

La naturaleza tiene cierta clase de sistema de coordenadas geométrico aritméticas, porque la naturaleza tiene todo tipo de modelos. Lo que nosotros concebimos de la naturaleza está en los modelos y todos los modelos de la naturaleza son tan bellos. Se me ocurre que el sistema de la naturaleza debe ser una verdadera belleza, porque en química encontramos que las asociaciones se presentan siempre en bellos números enteros —no existen fracciones.
Richard Buckminster Fuller

Sinopsis

- E.1 Introducción
- E.2 Cómo abreviar números binarios como octales y hexadecimales
- E.3 Cómo convertir números octales y hexadecimales a binarios
- E.4 Cómo convertir de binario, octal y hexadecimal a decimal
- E.5 Cómo convertir de decimal a binario, octal o hexadecimal
- E.6 Números binarios negativos: notación de complemento a dos

Resumen • Terminología • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.

E.1 Introducción

En este apéndice, presentamos los sistemas numéricos clave utilizados por los programadores de C, en especial cuando están trabajando sobre proyectos de software que requieren una cercana interacción a “nivel máquina”. Con el hardware proyectos como éstos incluyen sistemas operativos, software de redes de cómputo, compiladores, sistemas de base de datos y aplicaciones que requieran de alto rendimiento.

Cuando en un programa en C escribimos un entero como 227 o -63, se supone que el número está escrito en *sistema numérico decimal (de base 10)*. En el sistema numérico decimal los dígitos son 0, 1, 2, 3, 4, 5, 6, 7, 8, y 9. El dígito menor es 0 y el mayor es 9 —uno menos que la *base*, que es 10. En forma interna, las computadoras utilizan el *sistema numérico binario (de base 2)*. El sistema numérico binario utiliza únicamente dos dígitos, es decir 0 y 1. Su dígito menor es 0 y su dígito mayor es 1 —uno menos que la base de 2.

Como veremos, los números binarios tienden a ser mucho más largos que sus equivalentes decimales. Los programadores que tienen que trabajar en lenguajes de ensambladores, y en lenguajes de alto nivel como C que permiten a los programadores llegar hasta el “nivel máquina”, por lo que otros dos sistemas numéricos se han hecho populares el *sistema numérico octal (de base 8)* y el *sistema numérico hexadecimal (de base 16)* primordialmente debido a que resultan convenientes para abreviar números binarios.

En el sistema numérico octal, los dígitos tienen un rango del 0 al 7. Dado que tanto el sistema numérico binario como el sistema numérico octal tienen menos dígitos que el sistema numérico decimal, sus dígitos son los mismos que los correspondientes dígitos en decimal.

El sistema numérico hexadecimal presenta un problema, porque requiere de dieciséis dígitos —un dígito menor 0 y un dígito mayor, con un valor equivalente al 15 decimal (uno menos que la base de 16). Por regla convencional, utilizamos las letras A hasta F para representar los dígitos hexadecimales que corresponden a los valores decimales del 10 hasta el 15. Por lo tanto en hexadecimal podemos tener números como 876, que están formados únicamente por dígitos parecidos a los decimales, números como 8A55F, que están formados de dígitos y de letras, y números como FFE, que sólo están formados por letras. Ocasionalmente, un número hexadecimal deletrea una palabra común, como FACE, o bien FEED y esto pudiera parecer extraño a aquellos programadores acostumbrados a trabajar solo con números.

Cada uno de estos sistemas numéricos utilizan *notación posicional* —la posición en la cual se escribe un dígito tiene un *valor posicional diferente*. Por ejemplo, en el número decimal 937 (el 9, el 3 y el 7 se conocen como *valores simbólicos*), decimos que el 7 está escrito en la *posición de las unidades*, el 3 está escrito en la *posición de las decenas*, y el 9 está escrito en la *posición de las centenas*. Note que cada una de estas posiciones es una potencia de la base (base 10), y estas potencias empiezan en 0 y se incrementan en 1 conforme nos movemos hacia la izquierda dentro del número (figura E.3).

Dígito binario	Dígito octal	Dígito decimal	Dígito hexadecimal
0	0	0	0
1	1	1	1
	2	2	2
	3	3	3
	4	4	4
	5	5	5
	6	6	6
	7	7	7
		8	8
		9	9
			A (valor decimal 10)
			B (valor decimal 11)
			C (valor decimal 12)
			D (valor decimal 13)
			E (valor decimal 14)
			F (valor decimal 15)

Fig. E.1 Dígitos de los sistemas numéricos binario, octal, decimal y hexadecimal.

Atributo	Binario	Octal	Decimal	Hexadecimal
Base	2	8	10	16
Dígito menor	0	0	0	0
Dígito mayor	1	7	9	F

Fig. E.2 Comparación de los sistemas numéricos binario, octal, decimal y hexadecimal.

Valores posiciones en el sistema numérico decimal			
Dígito decimal	9	3	7
Nombre de la posición	Centenas	Decenas	Unidades
Valor posicional	100	10	1
Valor posicional como una potencia de la base (10)	10 ²	10 ¹	10 ⁰

Fig. E.3 Valores posicionales en el sistema numérico decimal.

Para números decimales más grandes, las siguientes posiciones a la izquierda serían la *posición de los millares* (10 a la tercera potencia), la *posición de los diez millares* (10 a la cuarta potencia), la *posición de los cien millares* (10 a la quinta potencia), la *posición de los millones* (10 a la sexta potencia), la *posición de los diez millones* (10 a la séptima potencia), y así en lo sucesivo.

En el número binario 101, decimos que el 1 más a la derecha está escrito en la *posición de las unidades*, el 0 está escrito en la *posición de los dos*, y el 1 más a la izquierda está escrito en la *posición de los cuatros*. Note que cada una de estas posiciones es una potencia de la base (base 2), y que estas potencias empiezan en 0 y se incrementan por 1 conforme nos movemos a la izquierda en el número (figura E.4).

Para números binarios más grandes, la siguiente posición a la izquierda sería la *posición de los ochos* (2 a la tercera potencia), la *posición de los diez y seis* (2 a la cuarta potencia), la *posición de los treinta y dos* (2 a la quinta potencia), la *posición de los sesenta y cuatro* (2 a la sexta potencia), y así en lo sucesivo.

En el número octal 425, decimos que el 5 está escrito en la *posición de las unidades*, el 2 está escrito en la *posición de los ochos*, y el 4 está escrito en la *posición de los sesenta y cuatro*. Note que cada una de estas posiciones es una potencia de la base (base 8), y que estas potencias empiezan en 0 y se incrementan en 1 conforme vamos hacia la izquierda en el número (figura E. 5).

Para números octales más grandes, las siguientes posiciones a la izquierda serían la *posición de los quinientos doces* (8 a la tercera potencia), la *posición de los cuatro mil noventa y seis* (8 a la cuarta potencia), la *posición de los treinta y dos mil setecientos y sesenta y ochos* (8 a la quinta potencia), y así en lo sucesivo.

En el número hexadecimal 3DA, decimos que la A está escrita en la *posición de las unidades*, la D está escrita en la *posición de los diez y seis*, y el 3 está escrito en la *posición de los doscientos cincuenta y seis*. Note que cada una de estas posiciones es una potencia de la base (base 16) y que estas potencias empiezan en 0 y se incrementan en 1 conforme vamos hacia la izquierda en el número (figura E.6).

Valores posicionales en el sistema numérico binario			
Dígito binario	1	0	1
Nombre de la posición	Cuatros	Dos	Unidades
Valor posicional	4	2	1
Valor posicional como una potencia de la base (2)	2 ²	2 ¹	2 ⁰

Fig. E.4 Valores posicionales en el sistema numérico binario.

Valores posicionales en el sistema numérico octal			
Dígito octal	4	2	5
Nombre de la posición	Sesenta y cuatros	Ochos	Unidades
Valor posicional	64	8	1
Valor posicional como una potencia de la base (8)	8 ²	8 ¹	8 ⁰

Fig. E.5 Valores posicionales en el sistema numérico octal.

Valores posicionales en el sistema numérico hexadecimal			
Dígito hexadecimal	3	D	A
Nombre de la posición	Doscientos cincuenta y seis	Diez y seis	Unidades
Valor posicional	256	16	1
Valor posicional como una potencia de la base (16)	16 ²	16 ¹	16 ⁰

Fig. E.6 Valores posicionales en el sistema numérico hexadecimal.

Para números hexadecimales mayores, las siguientes posiciones a la izquierda serían la *posición cuatro mil noventa y seis* (16 a la tercera potencia), la *posición treinta y dos mil setecientos y sesenta y ochos* (16 a la cuarta potencia), y así sucesivamente.

E.2 Cómo abreviar números binarios como octales y hexadecimales

El uso principal para los números octales y hexadecimales en la computación es para abreviar representaciones binarias largas. En la figura E.7 se destaca el hecho que números binarios largos pueden ser expresados en forma concisa en sistemas numéricos con bases más altas que el sistema numérico binario.

Número decimal	Representación binaria	Representación octal	Representación hexadecimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Fig. E.7 Equivalentes decimal, binario, octal y hexadecimal.

Una relación particular importante que tienen tanto el sistema numérico octal como el hexadecimal con el sistema binario, es que las bases de los sistemas octal y hexadecimal (8 y 16 respectivamente), son potencias de la base del sistema numérico binario (base 2). Considere el siguiente número binario de 12 dígitos y sus equivalentes octal y hexadecimal. Vea si puede determinar cómo resulta conveniente esta relación para abreviar números binarios a octal y hexadecimal. La respuesta sigue a los números.

Número binario	Equivalente octal	Equivalente hexadecimal
100011010001	4321	8D1

Para ver como los números binarios se convierten con facilidad a octal, sólo divida el número binario de 12 dígitos en grupos cada uno de ellos de tres bits consecutivos, y escriba dichos grupos sobre los dígitos correspondientes del número octal como sigue

100	011	010	001
4	3	2	1

Note que el dígito octal que usted ha escrito bajo cada número de estos tres bits corresponde precisamente al equivalente octal del número binario de tres dígitos, tal y como se muestra en la figura E.7. El mismo tipo de relación se puede observar en la conversión de números de binario a hexadecimal. En particular divida el número binario de 12 dígitos en grupos de cuatro bits consecutivos cada uno y escriba estos grupos sobre los dígitos correspondientes al número hexadecimal como sigue

1000	1101	0001
8	D	1

Note que el dígito hexadecimal que ha escrito bajo cada grupo de cuatro bits corresponde precisamente al equivalente hexadecimal de dicho número binario de cuatro dígitos, tal y como se muestra en la figura E.7.

E.3 Cómo convertir números octales y hexadecimales a binarios

En la sección anterior, vimos cómo convertir números binarios a sus equivalentes octal y hexadecimal formando grupos de dígitos binarios y sólo reescribiendo estos grupos en sus valores digitales octales o hexadecimales. Este proceso pudo ser utilizado en reversa para producir el equivalente binario de un número octal o hexadecimal dado.

Por ejemplo, el número octal 653 se convierte a binario simplemente escribiendo el 6 como su equivalente binario de 3 dígitos 110, el 5 como su equivalente binario de 3 dígitos 101, y el 3 como su equivalente binario de 3 dígitos 011 para formar el número binario de 9 dígitos 110101011.

El número hexadecimal FAD5 se convierte a binario simplemente escribiendo la F como su equivalente binario de 4 dígitos 1111, la A como su equivalente binario de 4 dígitos 1010, la D como su equivalente binario de 4 dígitos 1101, y el 5 como su equivalente binario de 4 dígitos 0101 para formar el número de 16 dígitos 1111101011010101.

E.4 Cómo convertir de binario, octal y hexadecimal a decimal

Dado que estamos acostumbrados a escribir en decimal, a menudo resulta conveniente convertir un número binario, octal o hexadecimal a decimal para darnos cuenta de lo que “realmente” vale ese número. Nuestros diagramas en la sección E.1 expresan los valores posicionales en decimal. Para convertir un número de decimal a otra base, multiplique el equivalente decimal de cada dígito por su valor posicional, y sume dichos productos. Por ejemplo, el número binario 110101 se convierte al decimal 53 tal y como se muestra en la figura E.8.

Cómo convertir un número binario a decimal						
Valores posicionales:	32	16	8	4	2	1
Valores simbólicos:	1	1	0	1	0	1
Productos:	1*32=32	1*16=16	0*8=0	1*4=4	0*2=0	1*1=1
Suma:	= 32 + 16 + 0 + 4 + 0 + 1 = 53					

Fig. E.8 Cómo convertir un número binario a decimal.

Para convertir el octal 7614 al decimal 3980, utilizamos la misma técnica, esta vez utilizando los valores posicionales octales apropiados, tal y como se muestra en la figura E.9.

Para convertir el valor hexadecimal AD3B al decimal 44347, utilizamos la misma técnica, esta vez utilizando los valores posicionales hexadecimales apropiados, como se muestra en la figura E.10.

E.5 Cómo convertir de decimal a binario, octal o hexadecimal

Las conversiones de la sección anterior son una consecuencia natural de las reglas convencionales de la notación posicional. La conversión de decimal a binario, octal o hexadecimal también sigue estas reglas convencionales.

Suponga que deseamos convertir el decimal 57 a binario. Empezamos escribiendo los valores posicionales de las columnas derecha a la izquierda hasta que alcanzamos una columna cuyo valor posicional es mayor que el número decimal. No necesitamos esta columna, por lo que la descartamos. Por lo tanto primero escribimos:

Cómo convertir un número octal a decimal				
Valores posicionales:	512	64	8	1
Valores simbólicos:	7	6	1	4
Productos:	7*512=3584	6*64=384	1*8=8	4*1=4
Suma:	= 3584 + 384 + 8 + 4 = 3980			

Fig. E.9 Cómo convertir un número octal a decimal.

Cómo convertir un número hexadecimal a decimal				
Valores posicionales:	4096	256	16	1
Valores simbólicos:	A	D	3	B
Productos:	A*4096=40960	D*256=3328	3*16=48	B*1=11
Suma:	= 40960 + 3328 + 48 + 11 = 44347			

Fig. E.10 Cómo convertir un número hexadecimal a decimal.

Valores posicionales: 64 32 16 8 4 2 1

A continuación descartamos la columna con el valor posicional 64 dejando:

Valores posicionales: 32 16 8 4 2 1

A continuación trabajamos partiendo de la columna más a la izquierda hacia la derecha. Dividimos 57 entre 32 y observamos que el resultado es 1 con un residuo de 25, por lo que en la columna de los 32 escribimos 1. Ahora dividimos 25 entre 16 y observamos que 25 entre 16 da 1 con un residuo de 9 y escribimos 1 en la columna de los 16. Ahora dividimos 9 entre 8 y observamos que 9 entre 8 da 1 con un residuo de 1. Las siguientes dos columnas cada una de ellas produce resultados de cero cuando dividimos 1 entre sus valores posicionales, por lo que escribimos ceros en las columnas 4 y 2. Finalmente 1 dividido entre 1 es 1, por lo que escribimos 1 en la columna de los unos. Esto da como resultado:

Valores posicionales: 32 16 8 4 2 1
Valores simbólicos 1 1 1 0 0 1

y, por lo tanto, el equivalente del decimal 57 en binario es 111001

Para convertir el decimal 103 a octal, empezamos escribiendo los valores posicionales de las columnas hasta que alcanzamos una columna cuyo valor posicional es mayor que el número decimal. No necesitamos de esa columna, por lo que la descartamos. Por lo tanto primero escribimos:

Valores posicionales: 512 64 8 1

A continuación descartamos la columna con el valor posicional 512, dando por resultado:

Valores posicionales: 64 8 1

A continuación trabajamos a partir de la columna más a la izquierda y hacia la derecha. Dividimos 103 entre 64 y observamos que 103 entre 64 da a uno con un residuo de 39, por lo que escribimos 1 en la columna de los 64. A continuación dividimos 39 entre 8 y observamos que el resultado es 4 con un residuo de 7, y escribimos 4 en la columna de los 8. Por último dividimos 7 entre 1 y observamos que el resultado es 7 sin residuo, por lo que escribimos 7 en la columna de los unos. Esto da como resultado:

Valores posicionales: 64 8 1
Valores simbólicos 1 4 7

y, por lo tanto, el decimal 103 es equivalente al octal 147.

Para convertir el decimal 375 a hexadecimal, empezamos escribiendo los valores posicionales de las columnas hasta que llegamos a una columna cuyo valor posicional es mayor que el número decimal. No necesitamos dicha columna, por lo que la descartamos. Entonces, primero escribimos

Valores posicionales: 4096 256 16 1

A continuación descartamos la columna con el valor posicional 4096, dando por resultado:

Valores posicionales: 256 16 1

A continuación trabajamos a partir de la columna más a la izquierda hacia la derecha. Dividimos 375 entre 256 y observamos que el resultado es uno con un residuo de 119, por lo que escribimos 1 en la columna de los 256. Ahora dividimos 119 entre 16 y observamos que el resultado es 7 con un residuo de 7 y escribimos 7 en la columna de los 16. Por último dividimos 7 entre 1 y observamos que existe un resultado de 7 sin residuo, por lo que escribimos 7 en la columna de los unos. Esto da como resultado:

Valores posicionales: 256 16 1
Valores simbólicos 1 7 7

por lo tanto, el equivalente del decimal 375 en hexadecimal es 177.

E.6 Números binarios negativos: notación de complemento a dos

El análisis en este apéndice ha sido enfocado a números positivos. En esta sección, explicaremos cómo representan las computadoras números negativos mediante la *notación de complemento a dos*. Primero explicaremos cómo se forma el complemento a dos de un número binario, y a continuación mostraremos porqué representa el valor negativo de un número binario dado.

Considere una máquina con enteros de 32 bits. Suponga

`int value = 13;`

La representación en 32 bits de `value` es

00000000 00000000 00000000 00001101

Para formar el negativo de `value`, primero formamos su complemento a uno aplicando el operador de complemento a nivel de bits (~) de C.

`ones_complement_of_value = ~value;`

Internamente, `~value` es ahora `value` con cada uno de sus bits invertidos —los unos se han convertido en ceros y los ceros se han convertido en unos, como sigue:

`value:`
00000000 00000000 00000000 00001101

`~value` (es decir, el complemento a uno de `value`):
11111111 11111111 11111111 11110010

Para formar el complemento a dos de `value`, simplemente añadimos uno al complemento a uno de `value`. Por lo tanto

El complemento a dos de `value`:
11111111 11111111 11111111 11110011

Ahora si esto de hecho es igual a -13, deberíamos estar en condiciones de añadirlo al número binario 13 y obtener como resultado 0. Probémoslo:

00000000 00000000 00000000 00001101
+11111111 11111111 11111111 11110011

00000000 00000000 00000000 00000000

Se descarta el bit de acarreo que resulta en la columna más a la izquierda y como resultado obtenemos cero. Si a un número le añadimos el complemento a uno del número, el resultado serían todos unos. La clave para obtener un resultado de todos ceros es que el complemento a dos es uno más que el complemento a uno. La adición de uno hace que cada columna añada a cero, con un uno que se acarrea a la izquierda. El acarreo se va trasladando hacia la izquierda hasta que se descarta del bit más a la izquierda, y de ahí el número resultante de todos ceros.

En realidad las computadoras llevan a cabo una substracción como

`x = a - value;`

sumando el complemento a dos de **value** a **a** como sigue:

x = a + (~value + 1);

Suponga que **a** es 27 y **value** es 13, como antes. Si el complemento a dos de **value** es de hecho el valor negativo de **value**, entonces la suma del complemento a dos de **value** a **a** debería producir el resultado 14. Probémoslo:

a (es decir, 27)	00000000 00000000 00000000 00011011
+ (~value + 1)	+11111111 11111111 11111111 11110011

	00000000 00000000 00000000 00001110

lo que realmente da igual a 14.

Resumen

- Cuando escribimos un entero como 19 y 227 ó -63 en un programa C, automáticamente se supone que el número está en sistema numérico decimal (en base 10). Los dígitos en sistema numérico decimal son 0, 1, 2, 3, 4, 5, 6, 7, 8, y 9. El dígito menor es cero y el dígito mayor es 9 —uno menos que la base de 10.
- En forma interna, las computadoras utilizan el sistema numérico binario (de base 2). El sistema numérico binario tiene sólo dos dígitos, es decir 0 y 1. Su número menor es 0 y su número mayor es 1 —uno menos que el de la base de 2.
- El sistema numérico octal (de base 8) y el sistema numérico hexadecimal (de base 16) se han hecho populares principalmente debido a que resultan convenientes para abreviar números binarios.
- Los dígitos del sistema numérico octal van desde el 0 al 7.
- El sistema numérico hexadecimal presenta un problema, porque requiere de 16 dígitos —un dígito menor 0 y un dígito mayor con un valor equivalente al 15 decimal (uno menos que la base de 16). Por regla convencional, utilizamos las letras A hasta F para representar los dígitos hexadecimales que corresponden a los valores decimales 10 hasta el 15.
- Cada sistema numérico utiliza notación posicional —cada posición en la cual se escribe un dígito, tiene un valor posicional diferente.
- Una relación particularmente importante que tienen tanto el sistema numérico octal como el hexadecimal con el sistema binario, es que las bases de los sistemas octal y hexadecimal (8 y 16 respectivamente) son potencias de la base del sistema numérico binario (base 2).
- Para convertir un número octal a un número binario, simplemente reemplace cada dígito octal por su equivalente binario de tres dígitos.
- Para convertir un número hexadecimal a un número binario, simplemente reemplace cada dígito hexadecimal por su equivalente binario de cuatro dígitos.
- Dado que estamos acostumbrados a trabajar en decimal, a menudo resulta conveniente convertir un número binario, octal o hexadecimal a decimal para tener una mejor idea de lo que en verdad vale un número.
- Para convertir un número de decimal desde otra base, multiplique el equivalente decimal de cada dígito por su valor posicional y sume dichos productos.
- Las computadoras representan los números negativos utilizando notación de complemento a dos.

- Para formar el negativo de un valor en binario, primero forme el complemento a uno aplicando el operador de complemento a nivel de bits (~) de C. Esto invierte los bits del valor. Para formar el complemento a dos de dicho valor, sólo añada uno al valor del complemento a uno.

Terminología

base	dígito
sistema numérico de base 2	sistema numérico hexadecimal
sistema numérico de base 8	valor negativo
sistema numérico de base 10	sistema numérico octal
sistema numérico de base 16	notación de complemento a uno
sistema numérico binario	notación posicional
operador de complemento a nivel de bits (~)	valor posicional
conversiones	valor simbólico
sistema numérico decimal	notación de complemento a dos

Ejercicios de autoevaluación

- E.1 Las bases de los sistemas numéricos decimal, binario, oct y hexadecimal son _____, _____, _____, y _____ respectivamente.
- E.2 En general, las representaciones decimal, octal y hexadecimal de un número binario dado contienen (más/menos) dígitos que los que contiene el número binario.
- E.3 (Verdadero/falso). Una razón popular para utilizar el sistema numérico decimal es que forma una notación conveniente para abreviar números binarios simplemente sustituyendo un dígito decimal por cada grupo de cuatro dígitos binarios.
- E.4 La representación (octal/hexadecimal/decimal) de un valor binario muy grande es la más concisa (de las alternativas dadas).
- E.5 (Verdadero/falso). El dígito mayor en cualquier base es uno más que la base.
- E.6 (Verdadero/falso). El dígito menor de cualquier base es uno menos que la base.
- E.7 El valor posicional del dígito más a la derecha de cualquier número en binario, octal, decimal o hexadecimal es siempre _____.
- E.8 El valor posicional del dígito a la izquierda del dígito más a la derecha de cualquier número en binario, octal, decimal o hexadecimal es siempre igual a _____.
- E.9 Complete los valores faltantes en esta tabla de valores posicionales para las cuatro posiciones más a la derecha en cada uno de los sistema numéricos indicados.
- | | | | | |
|-------------|------|-----|-----|-----|
| decimal | 1000 | 100 | 10 | 1 |
| hexadecimal | ... | 256 | ... | ... |
| binario | ... | ... | ... | ... |
| octal | 512 | ... | 8 | ... |
- E.10 Convertir el número binario 110101011000 a octal y a hexadecimal.
- E.11 Convertir el número hexadecimal FACE a binario.
- E.12 Convertir el número octal 7316 a binario.

- E.13** Convertir el número hexadecimal **4FEC** a octal. (*Sugerencia:* primero convierta **4FEC** a binario y a continuación convierta el número binario a octal).)
- E.14** Convierta el número binario **1101110** a decimal.
- E.15** Convierta el número octal **317** a decimal.
- E.16** Convierta el número hexadecimal **EFD4** a decimal.
- E.17** Convierta el número decimal **177** a binario, a octal, y a hexadecimal.
- E.18** Muestre la representación binaria del decimal **417**. Después muestre el complemento a uno de **417** y el complemento a dos.
- E.19** ¿Cuál es el resultado cuando el complemento a uno de un número se añade a sí mismo?

Respuestas a los ejercicios de autoevaluación

- E.1** 10, 2, 8, 16.
- E.2** Menos.
- E.3** Falso.
- E.4** Hexadecimal.
- E.5** Falso —el dígito mayor en cualquier base es uno menos que la base.
- E.6** Falso —el dígito menor de cualquier base es cero.
- E.7** 1 (la base elevada a la potencia cero).
- E.8** La base del sistema numérico.
- E.9** Complete los valores faltantes en esta tabla de valores posicionales para las cuatro posiciones más a la derecha en cada uno de los sistemas numéricos indicados.

decimal	1000	100	10	1
hexadecimal	4096	256	16	1
binario	8	4	2	1
octal	512	64	8	1

- E.10** Octal **6530**; hexadecimal **D58**.
- E.11** Binario **1111 1010 1100 1110**.
- E.12** Binario **111 011 001 110**
- E.13** Binario **0 100 111 111 101 100**; Octal **47754**
- E.14** Decimal **2+4+8+32+64=110**.
- E.15** Decimal **7+1*8+3*64=7+8+192=207**
- E.16** Decimal **4+13*16+15*256+14*4096=61396**.
- E.17** Decimal **177**
a binario:

256 128 64 32 16 8 4 2 1
128 64 32 16 8 4 2 1
(1*128)+(0*64)+(1*32)+(1*16)+(0*8)+(0*4)+(0*2)+(1*1)
10110001

a octal:

512 64 8 1
64 8 1
(2*64)+(6*8)+(1*1)
261

a hexadecimal:

256 16 1
16 1
(11*16)+(1*1)
(B*16)+(1*1)
B1

E.18 Binario:

512 256 128 64 32 16 8 4 2 1
256 128 64 32 16 8 4 2 1
(1*256)+(1*128)+(0*64)+(1*32)+(0*16)+(0*8)+(0*4)+(0*2)+(1*1)
110100001

Complemento a uno:**001011110**
Complemento a dos:**001011111**
Verificación: número binario original + su complemento a dos

110100001
001011111

000000000

E.19 Cero

Ejercicios

- E.20** Algunas personas alegan que muchos de nuestros cálculos serían más fáciles en un sistema numérico de base **12** porque **12** es divisible por muchísimos más números que **10** (debido a la base **10**). ¿Cuál es el dígito menor en la base **12**? ¿Cuál debería ser el símbolo más alto para el dígito en la base **12**? ¿Cuáles son los valores posicionales para las cuatro posiciones más a la derecha de cualquier número en un sistema numérico de base **12**?
- E.21** ¿Cómo se relaciona el valor simbólico más alto en los sistemas numéricos anteriormente analizados con el valor posicional del primer dígito a la izquierda del dígito más a la derecha de cualquier número en estos sistemas numéricos?
- E.22** Complete la tabla siguiente de valores posicionales para las cuatro posiciones más a la derecha en cada uno de los sistemas numéricos indicados:

decimal	1000	100	10	1
base 6	...	...	6	...
base 13	...	169	...	...
base 3	27	...	...	...

- E.23** Convierta el binario 100101111010 a octal y a hexadecimal.
- E.24** Convierta el hexadecimal a 3A7D a binario.
- E.25** Convierta el hexadecimal a 765F a octal. (*Sugerencia:* primero convierta 765F a binario y a continuación convierta dicho número binario a octal).
- E.26** Convierta el número binario 1011110 a decimal.
- E.27** Convierta el número octal 426 a decimal.
- E.28** Convierta el número hexadecimal FFFF a decimal.
- E.29** Convierta el número decimal 299 a binario, a octal y a hexadecimal.
- E.30** Muestre la representación binario del decimal 779. A continuación muestre el complemento a uno de 779, así como el complemento a dos.
- E.31** ¿Cuál es el resultado cuando el complemento a dos de un número se añade a sí mismo?
- E.32** Muestre el complemento a dos del valor entero -1 en una máquina con enteros de 32 bits.

INDICE

Indice

- !=, operador de desigualdad, 39
- # bandera, 377, 378
- # operador del preprocesador, 28, 527
- ## operador del preprocesador 527
- #define macro, 581
- #define constante simbólica, 575, 608
- #define directriz de preprocesador, 209, 523, 567
- #define NDEBUG, 859
- #endif directriz de preprocesador, 608
- #error directriz de preprocesador, 526
- #ifdef directriz de preprocesador, 526, 608
- #ifndef directriz de preprocesador, 526
- #include directriz de preprocesador, 209, 522
- #line directriz de preprocesador, 527
- #pragma directriz de preprocesador, 527
- #undef directriz de preprocesador, 525
- % operador módulo, 35
- %%, 372, 380
- %c, 217, 158, 371, 380
- %d, 30, 31, 158, 368, 380, 381
- %E, 370, 380
- %e, 370, 380
- %f, 74, 158, 370, 380, 381
- %G, 370, 380
- %g, 370, 380
- %hd, 158
- %i, 368, 379, 380
- %ld, 158, 175
- %Lf, 158
- %lf, 158
- %lu, 158
- %n, 372, 380
- %o, 368, 380
- %p, 262, 372, 380
- %s, 371, 380
- %u, 158, 367, 368, 380
- %X, 368, 380
- %x, 168, 380
- & operador de dirección, 30
- & operador AND para bits, 407
- &= operador de asignación AND, 414
- && operador, 123, 179
- '\o', 214
- '\f', 321
- '\n', 321
- '\r', 321
- '\t', 321
- '\v', 321
- * caracter de indirección, 384
- * operador de multiplicación, 34
- * argv [], 540
- * fgets, 872
- * this, 693
- + bandera, 377
- ++ operador, 80
- += asignación de adición, 77
- > operador apuntador a estructura, 399, 597, 606
- . operador miembro de estructura, 399, 597
- / OR inclusivo para bits, 407
- :: operador unario de resolución de alcance, 578, 586, 604
- :? 179
- < símbolo de redirección de entrada, 537
- << desplazamiento a la izquierda, 407
- << operador estándar de flujo de salida, 562
- <= operador de asignación de desplazamiento a la derecha, 414
- <= menor o igual a, 39
- = operador de igualdad, 39, 125
- >= mayor o igual a, 39
- >> símbolo de agregar salida, 537, 562
- >> desplazamiento a la derecha, 407
- >>= operador de asignación de desplazamiento a la izquierda, 414
- \ " secuencia de escape de comillas dobles, 379
- \ ' secuencia de escape de comilla sencilla, 26, 379
- \?, 379
- \\ secuencia de escape de diagonal invertida, 26, 379

\a alerta, secuencia de escape de, 26, 379
\b secuencia de escape, 379
\f forma continua, secuencia de escape de, 379
\n de pasar a la línea, secuencia de escape, 26, 379
\r de retorno de carro, secuencia de escape, 26, 379
\t tabulador horizontal, secuencia de escape de, 26, 379
\v secuencia de escape, 379
^ operador exclusivo OR para bits, 382, 407
+= operador de asignación exclusivo OR para bits, 414
~ operador de complemento a uno para bits, 407, 411, 603, 606, 617
DATE, constante simbólica predefinida, 528
FILE, constante simbólica predefinida, 528
LINE, constante simbólica predefinida, 528
STDC, constante simbólica predefinida, 528
TIME, constante simbólica predefinida, 528
| conducto, 537
|= operador de asignación inclusivo OR para bits, 414
||, 179
A
a modo de abrir archivo, 440
a+ modo de abrir archivo, 440,
a+ modo de actualizar archivo, 439
a.out, 11
ab modo de abrir archivo binario, 545
ab+ modo de abrir archivo binario, 545
abort, 528, 878
 abreviaturas similares al inglés, 7
 abrir tabla de archivo, 435
 abrir un archivo, 434
abs, 879
 abstracción, 151, 595, 731
 abstracción de datos, 642, 665, 755
 abstracción de datos en C, 21
 acceso inválido al almacenamiento, 547
 acción, 25, 26, 37, 56, 66
 acento circunflejo (^), 382
acos, 863
 acumulador, 305
 Ada, 10
 adición, 5
 administración dinámica de memoria, 260
 administrador de pantalla, 774, 781
 (ADT), tipo de datos abstractos, 601, 665, 666
 ADT, 665, 666
 agregados, 396
 agregar salida símbolo >, 537
 ajedrez, 252
 alcance, 168, 170
 alcance de archivo, 170, 605
 alcance de clase, 604, 605
 alcance de función, 170, 605
 alcance de función prototipo, 170, 171
 alcance del bloque, 170
 aleatoriedad, 163, 164
 alerta (' \a '), 26
 álgebra, 34
 algoritmo, 56, 66
 algoritmo completo, 59
 algoritmo rise-and-shine, 56
 alinear, 367
 almacenamiento automático, 169, 204
 almacenamiento libre, 570, 577
 almacenar, 517
 (ALU), unidad aritmética y lógica, 5
 ambiente local, 159
 American National Standards Committee on Computers and Information Processing, 8
 American National Standards Institute (ANSI), 3, 20, 846

amigo, 735
 ampersand (&), 30, 32
 análisis de datos de encuesta, 226, 229
 análisis de texto, 359
 ancho de campo, 112, 367, 372, 375, 384, 802, 814
 ancho de un campo de bits, 415
AND, 408
AND (&) para bits, 407, 409, 410, 427
 anidados, 35, 76
 anidamiento de estructura de control, 59
 ANSI, funciones estándar, 9
 ANSI C, 3, 9, 13, 231, 268
 ANSI C, comité, 155
 ANSI C, documento estándar, 8, 13
 ANSI C, biblioteca estándar, 149
 ANSI/ISO 9899: 1990, 8, 20
 añadir instrucción, 307, 517
 año bisiesto, 144
 Apéndice D, 112
 Apéndice E, Sistemas de numeración, 321, 328
 apilamiento de estructuras de control, 59
 aplicaciones comerciales, 9
 aplicaciones distribuidas
 cliente/servidor, 6
 apuntador, 260
 apuntador a apuntador, 472
 apuntador a un objeto, 607, 657
 apuntador a una función, 291
 apuntador a **void** (**void ***), 280, 470
 apuntador aritmético, 277, 279, 280, 358
 apuntador de archivo, 435
 apuntador de clase base, 738, 747, 773, 774, 780
 apuntador de clase base a objeto de clase derivada, 779
 apuntador de clase derivada, 734, 738, 747
 apuntador de expresión, 277
 apuntador de función, 292
 apuntador de notación, 281, 285

apuntador de posición de archivo, 442, 450
 apuntador **FILE**, 439
 apuntador genérico, 280
 apuntador **NULL**, 548
 apuntador **this**, 655, 684
 árbol, 37, 260, 396
 árbol binario, 468, 489
 árbol de búsqueda binaria, 490, 491, 494, 495, 505
 archivo, 432, 433
 archivo binario, 545
 archivo de acceso directo, 445, 446, 450
 archivo de acceso secuencial, 435
 archivo de cabecera, 28, 159, 523
 archivo de cabecera **ath.h**, 110, 111, 150, 159, 863
 archivo de cabecera de entrada/salida estándar (**stdio.h**), 28
 archivo de cabecera de matemáticas, 150
 archivo de cabecera personalizado, 160
 archivo de código fuente, 608
 archivo de entrada estándar, 434
 archivo de salida estándar, 434
 archivo de texto, 545
 archivo de transacción, 462
 archivo fuente, 607
 archivo maestro, 462
 archivo secuencial, 433, 441
 archivo temporal, 546
 archivos de cabecera estándar de biblioteca, 159, 522
 archivos fuente múltiples, 540, 542, 608
 área, 98
argc, 540
 argumento, 25, 149, 181, 523
 argumento por omisión, 578, 579, 581
 argumentos de línea de comando, 540
 argumentos más a la derecha por omisión, 578
 argumentos por omisión con constructores, 616
 argumentos variables, 867

atan, 863
atan2, 863
atexit, 543, 544, 878
atof, 325, 326, 876
atoi, 325, 326, 876
atol, 325, 326, 876
 atrapar una señal interactiva, 548
 atravesar un árbol binario, 492
 atributo, 595
 atributos de un objeto, 599
 audible (campana), 379
auto, 168
 autoasignación, 657, 693
 autodocumentar, 29, 397

B

B, 7
bad, función miembro, 827
badbit, 806, 827
 bandera 0 (cero), 377, 378
 bandera derecha, 819
 bandera espacio, 377
 bandera interna, 820
 bandera **ios::fixed**, 823, 825
 bandera **ios::scientific**, 823, 825
 bandera **left**, 819, 820
 bandera **lixed**, 823
 bandera mayúsculas, 823
 bandera **scientific**, 823, 824
 bandera **showbase**, 823
 bandera **showpoint**, 818, 819
 banderas, 367, 375, 376, 816
 base de datos, 434
BCPL, 7
 Bell Laboratories, 8; 13, 14, 560
 biblioteca de ejecución, 158
 biblioteca de manejo de funciones de caracteres, 321
 biblioteca de manejo de señales, 547
 biblioteca de utilerías generales (**stdlib**), 325, 875
 biblioteca **iostream**, 800
 bibliotecas de clase, 631, 731, 748
 bibliotecas estándar, 12

bifurcación, 511
bifurcación cero, 511, 512, 514, 515
bifurcación incondicional, 516, 548
bifurcación negativa, 511
binario, 143, 144, 321
bit, 432
bits de estado, 806
bloque, 25, 65, 154
bloque de construcción anidado, 131
bloque de control de archivo (FCB), 435, 436
bloque de datos, 344
bloque exterior, 171
bloque interno, 171
bloques constructivos apilados, 131
Bohm, C, 58
"bombardear", 70
Borland C++, 10, 222, 270
borrar árbol binario, 504
borrar lista enlazada, 182
borrar un nodo de una lista, 478
borrar un registro,
break, 115, 116, 120, 121, 144
bsearch, 879
búfer de salida, 829
buscar, 228
buscar funciones de la biblioteca de manejo de cadenas, 338
buscar un árbol binario, 495
buscar una lista enlazada, 182
búsqueda binaria, 182, 198, 228, 231, 232, 234, 257
búsqueda lineal, 182, 228, 230, 257
búsqueda recursiva de una lista, 504
Byron, Lord, 10
byte, 433

C
C, 8
C, compilador, 25
C, entorno, 11

C, lenguaje, 8
C, biblioteca, 21
C, preprocesador, 28, 522
C, biblioteca estándar, 8, 10, 25, 148, 160
C, con clases, 560
C y C++, palabras reservadas, 570
C *Answer Book*, 21
C++, 3, 6, 10, 14, 21, 155, 560
C++, palabras reservadas, 570
C++, comentario en una sola línea, 561
C++, flujo de entrada/salida, 562
cabecera de argumentos variables **stdarg.h**, 537
cabecera de función, 155
cabecera de una cola, 468, 484
cadena, 25, 690
cadena de caracteres, 25, 207
cadena de control de formato, 30, 31, 367, 375, 379
cadena es un apuntador, 319
cadena terminada en nulo, 805
caja tipográfica por omisión, 112, 115, 116
cálculos, 5, 31, 41
cálculos monetarios, 111
calendario, 144
calloc, 548, 878
campana, 26
campo, 433
campo de bit sin nombre, 415
campo de bit sin nombre de ancho cero, 415
campo de bits, 414, 415, 417
cantidad escalar, 220
cara o cruz, 197
carácter, 433
carácter de apóstrofes sencillos (`'`), 371
carácter de avance de hoja (`'\f'`), 321
carácter de comillas dobles (`"`), 26
carácter de escape, 25, 30, 377
carácter de interrogación (`?`), 379
carácter de relleno, 812, 814, 821

carácter de supresión de asignación (`*`), 384
carácter nulo, 214, 319
carácter terminador **NULL**, 214, 216, 320, 334, 335, 371
carácter tilde (`~`), 603
caracteres de barrido, 380
caracteres de control, 321
caracteres de espacio en blanco, 60, 806
caracteres de relleno, 819, 821, 822
caracteres especiales, 318
caracteres literales, 367
cargador, 11, 12
cargar, 10, 309, 517
cargar, 12
cargar un programa en memoria, 305
cartas urgiendo el pago, 361
casino, 160, 165
casino de juego, 160
casos (s) base, 173
catch, 570
cc, comando, 523
cc, comando, 12
ceil, función, 151, 864
Celsius, 392
ceros colgantes, 369, 818, 819
cerr, 800, 801
char, 158, 319
char*, 371
ciclar, 102
ciclar controlado por contador, 76
ciclo, 102, 69
ciclo controlado por contador, 78
ciclo de conteo, 105
ciclo infinito, 65, 73, 107
cin, 562, 800, 801, 809
cin.eof(), 809
circunferencia, 98
claridad, 13, 771
claridad del programa, 2, 13, 771
clase, 596
clase abstracta, 772
clase **Array**, 694
clase base, 730, 732

clase base abstracta, 772, 773, 774, 783, 785
clase base directa, 743
clase base indirecta, 743
clase base **ios**, 801, 818
clase base privada, 743
clase base protegida, 743
clase base pública, 743, 758
clase **BirthDate**, 750
clase **Boss**, 774, 777
clase **Circle**, 734, 735, 736, 737, 746, 771
clase **CommissionWorker**, 733, 774, 777, 778
clase **Complex**, 721, 722
clase **Cube**, 772
clase **Cylinder**, 754, 755
clase **Date**, 712
clase de almacenamiento, 168
clase de entero grande, 724
clase de pila de plantilla, 669
clase de plantilla, 668
clase derivada, 605, 731, 732
clase **Employee**, 740, 750, 774
clase **friend**, 650
clase **HourlyWorker**, 741, 774, 778, 782
clase **ios**, 825
clase **iostream**, 800
clase iterador, 774
clase **ostream**, 800
clase **PieceWorker**, 774, 778
clase **Point**, 734, 735, 785
clase **Quadrilateral**, 732, 773
clase **rationalNumber**, 725
clase **Rectangle**, 732, 771, 773
clase **Shape**, 772, 785
clase **Square**, 771
clase **String**, 700, 705
clase **string**, 700, 705
clase **TelephoneNumber**, 750
clase **ThreeDimensional-Object**, 772, 773
clase **time**, 609, 644, 885
clase **Triangle**, 771
clase **TwoDimensional-Object**, 772, 773
clases, 596
clases concretas, 772, 773

clases contenedor, 668, 690
clases de colecciones, 668
clasificación quiksort, 182, 312
clasificación sinking, 223
clasificar, 223
clasificar por selección, 182, 255
clave Morse, 361, 362
clave Morse internacional, 362
clear, función miembro, 827
clearerr, 875
cliente de una función, 566
clock, 885
clock_t, 884
clog, 800, 802
COBOL (Common Business Oriented Language), 9
código de caracteres, 338
código de lenguaje de máquina, 11
código de operación, 510
código fuente propietario, 748
código objeto, 11, 12
código optimizado, 518
código portátil, 8
coerción de argumentos, 157
cola de espera, 260, 396, 468, 484, 667
columna, 231
coma (`,`), operador, 107, 179
comando, 507
combinación de teclas de fin de archivo, 536
comentario, 25
comentario de una sola línea, 561
comillas doble, 32, 371
comisión, 247
cómo construir su propia computadora, 305
cómo construir su propio compilador, 509
cómo encadenar llamadas de funciones de miembros, 658
comparaciones $\log_2 n$, 495
comparar estructuras, 398
comparar uniones, 405
compilación, 12
compilación condicional, 522, 525
compilador, 4, 7, 11, 13, 25, 26, 28, 540

compilador optimizado, 169
compilar, 10, 11,
complejidad exponencial, 179
complemento, 411
complemento a uno, 901
complemento operador (`~`), 407
componente reusable, 732
componente reusable estandarizado, 732
componentes, 14
comportamiento, 599
comportamiento polimórfico, 780
comportamientos de un objeto, 595
composición, 605, 648, 750
composición a comparación de herencia, 749
computación conversacional, 31
computación distribuida, 6
computación interactiva, 31
computación personal, 6
computadora, 4
Computadora Apple, 6
computadora personal, 4, 748
Comunicaciones del ACM, 58
concatenación de cadenas, 358
concatenar el operador homónimo `<<`, 805
concatenar **puts**, 805
Concurrent C, 13, 20
condición, 37, 118
condición de continuación de ciclo, 102, 104, 105, 107, 118
condición simple, 122
condiciones de error, 159
conducir, 537
conducto (`|`), 537
conjunto de barrido, 382, 383
conjunto de barrido inverso, 382, 383
conjunto de caracteres, 338, 433
conservación de almacenamiento, 415
const, 222, 268, 271, 643
const, función de miembro, 643, 644
const, objeto, 643, 644, 648
const, calificador, 268, 574
constante, 509, 847
constante con nombre, 574
constante de cadena, 318